

ABSTRACT

Phishing attacks continue to pose a significant threat to modern digital systems, targeting users across various domains such as online banking, e-commerce, and social media platforms. These attacks involve the creation of fraudulent websites or deceptive URLs that closely resemble legitimate platforms in order to trick users into revealing sensitive information, including login credentials, financial details, and personal data. With the rapid advancement of internet technologies, phishing techniques have also evolved, becoming more sophisticated and difficult to detect using conventional methods.

Traditional phishing detection techniques, such as blacklist-based filtering and heuristic rule-based systems, have been widely used in the past. While these approaches are effective in identifying known phishing websites, they suffer from a critical limitation: the inability to detect newly generated or zero-day phishing attacks. Attackers frequently modify domain names, URL structures, and webpage content to evade detection mechanisms, rendering static approaches insufficient for real-world applications.

Machine learning-based detection methods were introduced to improve adaptability and detection accuracy by learning patterns from historical data. These methods utilize features such as URL length, number of subdomains, presence of suspicious characters, and domain-related information to classify websites. Although machine learning models provide better generalization compared to traditional approaches, they rely heavily on manual feature engineering, which requires domain expertise and extensive preprocessing. Furthermore, these models may struggle to

identify complex phishing patterns that closely mimic legitimate websites.

Recent advancements in deep learning have enabled the development of more powerful phishing detection systems capable of automatically learning meaningful feature representations from raw data. Models such as Convolutional Neural Networks (CNN) and Long Short-Term Memory (LSTM) networks have demonstrated strong performance in capturing structural and sequential patterns within URLs. CNN models are particularly effective in identifying local structural irregularities, such as suspicious symbols and abnormal character combinations, while LSTM-based models capture sequential dependencies and contextual relationships between different parts of the URL.

However, relying on a single deep learning architecture may not fully capture the diverse characteristics of phishing URLs. Different models specialize in different aspects of pattern recognition, and a standalone approach may result in incomplete feature representation. Additionally, deep learning models often suffer from issues such as overfitting and unstable predictions when used for final classification.

To overcome these limitations, this research proposes a hybrid phishing detection framework that integrates deep learning-based feature extraction with ensemble machine learning-based classification. The proposed system combines Convolutional Neural Networks (CNN) and Bidirectional Long Short-Term Memory (BiLSTM) networks to extract both structural and sequential features from URL data. The CNN component captures local patterns and structural anomalies, while the BiLSTM network analyzes the sequential flow and contextual dependencies within the URL.

Instead of directly using a neural network output layer for classification, the extracted deep features are fed into an Extreme Gradient Boosting (XGBoost) classifier. XGBoost is a powerful ensemble learning algorithm that builds multiple decision trees sequentially to improve prediction accuracy and reduce errors. By leveraging the strengths of both deep learning and ensemble learning, the proposed system achieves improved detection performance and greater prediction stability.

The proposed framework is designed to operate in a multi-stage pipeline. Initially, raw URL data is preprocessed and converted into a numerical format suitable for model input. The processed data is then passed through the CNN-BiLSTM architecture to generate deep feature representations. These features are subsequently used by the XGBoost classifier to produce the final prediction, categorizing the URL as either phishing or legitimate.

Experimental evaluation of the proposed model demonstrates its effectiveness in detecting phishing websites with high accuracy, precision, recall, and F1-score. The hybrid architecture significantly reduces false positive and false negative rates compared to traditional machine learning and standalone deep learning models. Furthermore, the system shows strong generalization capability, enabling it to detect previously unseen phishing URLs.

To validate its practical applicability, the proposed model is integrated into a browser extension-based environment for real-time phishing detection. The system monitors user browsing activity, analyzes visited URLs, and provides immediate warnings when a phishing website is detected. This real-time capability ensures enhanced user protection

without affecting browsing performance.

In conclusion, the proposed hybrid CNN–BiLSTM–XGBoost framework provides a robust, scalable, and efficient solution for phishing detection in modern web environments. By combining deep feature extraction with ensemble classification, the system effectively addresses the limitations of existing approaches and offers a reliable defense against evolving phishing threats. Future enhancements may include the integration of webpage content analysis, visual phishing detection, and advanced transformer-based architectures to further improve system performance and adaptability.

Keywords: Phishing Detection, Cybersecurity, Deep Learning, Convolutional Neural Network (CNN), Bidirectional LSTM (BiLSTM), XGBoost, URL Analysis, Machine Learning, Hybrid Model, Real-Time Detection

ACKNOWLEDGEMENT

We express our sincere gratitude to the Almighty for bestowing upon us the strength, wisdom, and determination to undertake and successfully complete this project. This accomplishment would not have been possible without the support, encouragement, and guidance of several individuals to whom we are deeply indebted.

We are profoundly grateful to our project supervisor, Mr. R. Gobinath , Assistant Professor, Department of Artificial Intelligence and Data Science, E.G.S. Pillay Engineering College, Nagapattinam, for her unwavering guidance, constructive feedback, and scholarly insights throughout the course of this project. Her expertise and encouragement have been invaluable in shaping the direction and quality of this work.

We extend our heartfelt thanks to the Head of the Department and all the faculty members of the Department of Artificial Intelligence and Data Science for their continued academic support and the stimulating environment they provide. We are also grateful to the Management of E.G.S. Pillay Engineering College for providing the necessary infrastructure and resources that facilitated the successful execution of this project.

We wish to acknowledge our fellow classmates and friends for their encouragement, valuable suggestions, and camaraderie throughout this endeavour. Finally, we are immensely thankful to our families for their unconditional love, patient understanding, and moral support, which sustained us through every challenge encountered during this project.

MOHAMED SAFWAN S/SOHANLAL S

TABLE OF CONTENTS

CHAPTER	TITLE	PAGE NO.
	ABSTRACT	(i)
	ACKNOWLEDGEMENT	(ii)
	TABLE OF CONTENTS	(iii)
	LIST OF FIGURES	(v)
	LIST OF TABLES	(vi)
	ABBREVIATIONS	(vii)
	NOMENCLATURE	(viii)
1	INTRODUCTION	1
	1.1 Background and Motivation	1
	1.2 Considerations in Existing Systems	2
	1.3 Problem Statement	4
	1.4 Proposed Solution and Key Contributions	6
	1.5 Organisation of the Report	8
2	LITERATURE REVIEW	10
	2.1 Rule-Based and Blacklist-Based Detection	10
	2.2 Machine Learning-Based Approaches	12
	2.3 Deep Learning-Based models	14
	2.4 Hybrid Detection Frameworks	16
	2.5 Research Gaps and Motivation	18
3	PROPOSED SYSTEM ARCHITECTURE	20
	3.1 Architectural Design Principles	20
	3.2 High-Level System Overview	22
	3.3 URL Preprocessing and Representation	24
	3.4 CNN-Based Structural Feature Extraction	26
	3.5 BiLSTM-Based Sequential Learning	28

	3.6 Hybrid Feature Fusion	30
	3.7 XGBoost-Based Classification	32
	3.8 Architectural Advantages	34
4	METHODOLOGY	36
	4.1 URL Data Collection Strategy	36
	4.2 Data Preprocessing and Representation	38
	4.3 CNN-Based Feature Extraction	42
	4.4 BiLSTM-Based Sequential Learning	46
	4.5 Hybrid CNN–BiLSTM–XGBoost Model	50
	4.6 Proposed Phishing Detection System	54
	4.7 Algorithmic Summary	60
5	EXPERIMENTAL SETUP AND EVALUATION	63
	5.1 Experimental Research Design	65
	5.2 Dataset Description and Sources	67
	5.3 Training Configuration	69
	5.4 Performance Metrics	72
	5.5 Results and Performance Analysis	75
	5.6 Zero-Day Detection Analysis	75
	5.7 Scalability and Efficiency Analysis	77
6	CONCLUSION AND FUTURE WORK	80
	6.1 Summary of Contributions	80
	6.2 Limitations	82
	6.3 Future Work	84
	REFERENCES	86

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
3.1	Overall Architecture of CNN–BiLSTM–XGBoost Model	23
3.2	Structure of URL Components	25
4.1	Data Collection from Multiple Sources	37
4.2	URL Encoding and Representation	40
4.3	CNN–BiLSTM Feature Extraction Pipeline	44
4.4	Hybrid Model Workflow	48
4.5	Data Collection Workflow	55
4.6	Data Preprocessing Pipeline	56
4.7	Encoded URL Representation	57
4.8	CNN Feature Extraction	58
4.9	BiLSTM Sequential Learning	59
4.10	Feature Fusion Process	60
4.11	XGBoost Classification	61
5.1	Dataset Distribution	66
5.2	Accuracy Comparison	68
5.3	Precision–Recall Graph	70
5.4	F1 Score Comparison	72
5.5	Confusion Matrix	73
5.6	Feature Contribution Analysis	75
5.7	Zero-Day Detection Rate	76
5.8	Scalability Analysis	78
5.9	False Positive Rate	79

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
2.1	Comparative Analysis of Phishing Detection Methods	19
4.1	Algorithmic Steps of Proposed System	60
5.1	Experimental Setup Configuration	64
5.2	Baseline Methods Comparison	66
5.3	Evaluation Metrics	69
5.4	Model Performance Comparison	72
5.5	System Performance Analysis	74
5.6	Feature Contribution and Characteristics	78

ABBREVIATIONS AND NOMENCLATURE

CNN	Convolutional Neural Network
BiLSTM	Bidirectional Long Short-Term Memory
XGBoost	Extreme Gradient Boosting
URL	Uniform Resource Locator
ML	Machine Learning
DL	Deep Learning
AI	Artificial Intelligence
TF	Term Frequency
IDF	Inverse Document Frequency
TF-IDF	Term Frequency – Inverse Document Frequency
RNN	Recurrent Neural Network
LSTM	Long Short-Term Memory
ReLU	Rectified Linear Unit
FPR	False Positive Rate
TPR	True Positive Rate
FNR	False Negative Rate
ROC	Receiver Operating Characteristic
AUC	Area Under Curve
API	Application Programming Interface
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure

NOMENCLATURE

SYMBOL	DESCRIPTION
D	Dataset of URLs: $D = \{(u_1, y_1), (u_2, y_2), \dots, (u_n, y_n)\}$
u_i	Individual URL instance
y_i	Class label of URL (0: Legitimate, 1: Phishing)
$f(u_i)$	Classification function mapping URL to output
U	Character sequence representation of URL
c_i	Character at position i in URL
x_i	Numerical encoding of character c_i
X	Encoded sequence representation
U_norm	Normalized URL after preprocessing
F_cnn	Feature vector extracted using CNN
F_bilstm	Feature vector extracted using BiLSTM
F_h	Hybrid feature representation (CNN + BiLSTM)
$\hat{y}_i$	Predicted output for URL u_i
TP	True Positives
TN	True Negatives
FP	False Positives
FN	False Negatives
Accuracy	Overall classification accuracy
Precision	Correct phishing predictions
Recall	Detection rate of phishing URLs
F1 Score	Balance between precision and recall
FPR	False Positive Rate
TPR	True Positive Rate
RT(N)	Response time for dataset size N
N	Total number of URLs
k	Number of selected features ($k \ll N$)
$\sigma(\cdot)$	Sigmoid activation function
ReLU(x)	Rectified Linear Unit activation

CHAPTER 1

INTRODUCTION

1.1 BACKGROUND AND MOTIVATION

In the modern digital era, the rapid growth of internet-based services such as online banking, e-commerce platforms, and social media networks has significantly increased the risk of cyber threats. Among these threats, phishing attacks have emerged as one of the most dangerous and widely Enormous forms of cybercrime.

Phishing involves the creation of fraudulent websites or URLs that closely resemble legitimate platforms, with the intention of deceiving users into revealing sensitive information such as login credentials, financial data, and personal details. As highlighted in the CNN-BiLSTM research paper , attackers exploit human trust and visual similarity to trick users, making phishing detection a critical concern in cybersecurity.

Traditional phishing detection techniques relied heavily on blacklist databases and rule-based filtering mechanisms. While these approaches were effective in identifying previously known phishing URLs, they fail to detect newly generated or zero-day attacks. The increasing sophistication of attackers, including techniques such as URL obfuscation, domain spoofing, and dynamic content generation, has made conventional methods inadequate. According to the Scientific Reports paper , modern phishing attacks are highly adaptive and continuously evolve to bypass static detection systems, leading to higher false positives and reduced reliability.

To address these challenges, advanced machine learning and deep learning techniques have been introduced. Deep learning models are capable of automatically learning complex patterns from data without requiring manual feature engineering.

In particular, hybrid models combining Convolutional Neural Networks (CNN) and Bidirectional Long Short-Term Memory (BiLSTM) networks have shown promising results in capturing both structural and sequential characteristics of URLs. CNN focuses on identifying spatial patterns such as suspicious characters and structures, while BiLSTM captures contextual relationships within sequences, enabling more accurate phishing detection.

The motivation behind this research is to develop a robust, scalable, and intelligent phishing detection system that overcomes the limitations of traditional methods. By integrating deep learning architectures with optimized feature extraction and classification techniques, the proposed system aims to provide high detection accuracy, low false positive rates, and real-time applicability. This work leverages the strengths of both referenced studies, combining deep URL intelligence concepts with hybrid CNN–BiLSTM architecture to build an efficient and adaptive phishing detection framework.

1.2 CONSIDERATIONS IN EXISTING SYSTEMS

Existing phishing detection systems exhibit several limitations when deployed in real-world environments, particularly in handling modern and evolving cyber threats. Traditional approaches such as blacklist-based filtering and rule-based detection rely heavily on previously identified phishing URLs, making them ineffective against newly generated or zero-day attacks. As discussed in the CNN–BiLSTM research , attackers continuously modify URL structures, domain names, and webpage layouts to bypass detection mechanisms, which significantly reduces the effectiveness of static detection systems.

Another critical limitation is the dependency on manual feature engineering in conventional machine learning models.

Features such as URL length, number of subdomains, and presence of special characters are manually extracted and used for classification. While these features provide some level of detection capability, they fail to capture the complex and hidden patterns present in modern phishing attacks. According to the Scientific Reports study , traditional models struggle with non-linear relationships and lack the ability to adapt dynamically to evolving phishing behaviors.

Furthermore, many existing systems face challenges related to scalability and computational efficiency. Large-scale environments such as enterprise networks and web platforms generate massive volumes of URL data, requiring detection systems to operate in real time.

However, applying complex models directly on large datasets increases computational overhead and latency, making them impractical for real-time deployment. This highlights the need for optimized architectures that balance accuracy and efficiency.

Another important issue is the high false positive rate observed in many detection systems. Incorrectly classifying legitimate websites as phishing can negatively impact user trust and system usability. As noted in the ADUIN framework , achieving a balance between detection accuracy and false positive reduction remains a major challenge in cybersecurity systems.

Additionally, most traditional systems lack the ability to integrate multi-dimensional data sources, such as lexical, host-based, and structural features of URLs. This limitation restricts the model's ability to gain a comprehensive understanding of phishing patterns.

Modern phishing attacks often combine multiple evasion techniques, which cannot be effectively detected using single-dimensional analysis. These limitations collectively highlight the need for an advanced, adaptive, and

intelligent phishing detection system that can overcome the shortcomings of existing approaches by leveraging deep learning and hybrid architectures.

1.2.1 Dependency on Static Blacklists

One of the most common approaches in traditional phishing detection systems is the use of static blacklist databases, which store previously identified malicious URLs. These systems compare incoming URLs with stored entries and block access if a match is found. While this method is simple and computationally efficient, it suffers from a major limitation: it can only detect known phishing websites. In real-world scenarios, attackers continuously generate new domains and URLs to bypass these blacklists, making them ineffective against zero-day phishing attacks.

Furthermore, maintaining and updating blacklist databases requires continuous monitoring and manual intervention, which introduces latency in detecting new threats. As phishing attacks evolve rapidly, relying solely on static blacklists creates a significant security gap, leaving users vulnerable to newly emerging malicious websites that are not yet recorded in the database.

1.2.2 Lack of Adaptive Learning Mechanisms

Traditional rule-based detection systems operate using predefined heuristics, such as identifying suspicious keywords, abnormal domain patterns, or irregular URL structures. Although these rules provide a basic level of protection, they lack the ability to adapt dynamically to new phishing techniques. Cyber attackers constantly modify their strategies by introducing new patterns, encoding techniques, and obfuscation methods to evade detection.

Since rule-based systems do not learn from new data, they quickly become outdated and ineffective. This rigid nature limits their ability to handle complex and evolving phishing behaviors. Additionally, creating and maintaining these rules requires domain expertise and continuous updates, making the system less

scalable. Therefore, the absence of adaptive learning capabilities significantly reduces the effectiveness of traditional phishing detection approaches in modern cybersecurity environments.

1.2.3 Limitations of Traditional Machine Learning

While machine learning-based approaches improve detection performance compared to rule-based systems, they introduce their own set of challenges. These models rely heavily on manual feature engineering, where domain-specific features such as URL length, number of subdomains, and presence of special characters are extracted and used for classification. Although these features provide useful information, they often fail to capture deeper and more complex relationships within the data.

As phishing websites become more sophisticated, attackers design URLs that closely resemble legitimate ones, making it difficult for traditional models to distinguish between them. Moreover, the performance of these models depends on the quality and relevance of the selected features, which may vary across different datasets. This dependency limits the generalization capability of the model, reducing its effectiveness in detecting unseen phishing attacks.

1.2.4 Incomplete Pattern Understanding in Single-Model Approaches

Most existing phishing detection systems rely on a single-model architecture, which focuses on capturing only one aspect of the data. For instance, some models are designed to analyze structural patterns in URLs, such as unusual symbols or domain arrangements, while others focus on sequential patterns, such as the order and relationships between characters. However, phishing attacks typically involve a combination of both structural and contextual manipulations.

A single-model approach is often insufficient to capture this complexity, leading to incomplete pattern understanding and reduced detection accuracy. This

limitation highlights the need for hybrid models that can integrate multiple learning techniques to capture diverse characteristics of phishing behavior. By combining different models, it is possible to achieve a more comprehensive understanding of malicious patterns and improve overall detection performance.

1.2.5 Challenges in Real-Time Deployment

Deploying phishing detection systems in real-world environments requires them to operate efficiently under high data volumes and real-time constraints. However, many advanced detection models are computationally intensive and require significant processing power, making them unsuitable for applications such as browser extensions or live web filtering systems. High latency in detection can negatively impact user experience and reduce the effectiveness of the system.

Additionally, as the number of users and web traffic increases, the system must be scalable enough to handle large datasets without performance degradation. Balancing accuracy, speed, and scalability remains a major challenge in phishing detection research. Systems must be optimized to deliver fast and reliable predictions while maintaining high detection accuracy, which is essential for practical deployment in cybersecurity applications.

1.2.6 High False Positive and False Negative Rates

Another critical issue in existing phishing detection systems is the occurrence of false positives and false negatives. A false positive occurs when a legitimate website is incorrectly classified as phishing, which can disrupt user experience and reduce trust in the system. On the other hand, a false negative occurs when a phishing website is mistakenly classified as legitimate, posing a serious security risk. Achieving a balance between these two types of errors is essential for building a reliable detection system.

Many traditional models struggle to maintain this balance, especially when dealing with complex and highly similar URL structures. Reducing error rates requires advanced models that can accurately distinguish between subtle differences in legitimate and malicious URLs. Therefore, improving classification accuracy while minimizing false predictions remains a key challenge in the development of effective phishing detection systems.

1.2.7 Lack of Multi-Dimensional Feature Integration

Modern phishing attacks often involve multiple layers of complexity, including lexical features, host-based characteristics, and structural patterns. However, many existing systems analyze only a limited set of features, leading to incomplete understanding of phishing behavior. For example, focusing solely on lexical features may overlook important host-based indicators such as domain age or DNS reputation.

Similarly, ignoring structural patterns may result in missing critical clues related to URL organization. The lack of multi-dimensional feature integration limits the ability of detection systems to identify sophisticated phishing attacks. Incorporating diverse feature sets can significantly enhance detection performance by providing a holistic view of the data. Therefore, there is a need for systems that can integrate multiple feature types and leverage them effectively for accurate classification.

1.3 PROBLEM STATEMENT

The rapid advancement of internet technologies and the widespread adoption of online services have significantly increased the exposure of users to various cybersecurity threats, among which phishing attacks remain one of the most critical and persistent challenges. Phishing websites are designed to mimic legitimate platforms in order to deceive users into revealing sensitive information

such as login credentials, financial details, and personal data. Despite the availability of multiple detection techniques, existing systems fail to provide a comprehensive solution capable of addressing the dynamic and evolving nature of phishing attacks. This creates an urgent need for an intelligent system that can accurately identify both known and unknown phishing websites with high reliability.

From a computational perspective, the phishing detection problem can be formulated as a binary classification task, where the objective is to classify a given URL as either legitimate or phishing. Let the dataset of URLs be represented as:

$$\mathbf{U} = \{u_1, u_2, u_3, \dots, u_n\} = \{u_1, u_2, u_3, \dots, u_n\}$$

where each u_i denotes an individual URL instance. The goal is to design a classification function:

$$f(u_i) \rightarrow \{0, 1\}$$

$f(u_i) = 0$ indicates a legitimate website,

$f(u_i) = 1$ indicates a phishing website.

The effectiveness of this classification function depends on its ability to learn meaningful patterns from the input data and generalize well to unseen examples. However, achieving this objective is highly challenging due to several inherent complexities associated with phishing attacks.

One of the primary challenges is the presence of highly obfuscated and dynamically generated URLs, where attackers use advanced techniques such as encoding, random token insertion, subdomain manipulation, and URL redirection to hide malicious intent. These techniques significantly reduce the effectiveness

of traditional detection methods, as they rely on static patterns and predefined rules. Furthermore, phishing URLs often exhibit both structural characteristics (such as unusual symbols and domain patterns) and sequential dependencies (such as the arrangement of characters and tokens), making it difficult for single-model approaches to capture the complete behavior of the attack.

Another major challenge lies in the requirement for real-time detection and scalability. Modern web environments generate massive volumes of URL data, requiring detection systems to process information quickly and efficiently. High computational complexity and latency can limit the practical deployment of detection models, especially in applications such as browser extensions and enterprise security systems. Therefore, the proposed solution must ensure a balance between high accuracy, low latency, and scalability.

In addition to detection accuracy, minimizing false positives and false negatives is equally important. A high false positive rate can lead to legitimate websites being incorrectly blocked, negatively impacting user experience and trust. On the other hand, false negatives can allow phishing websites to bypass detection, resulting in serious security risks. Therefore, the classification model must be designed to achieve optimal performance across multiple evaluation metrics, including accuracy, precision, recall, and F1-score.

Another limitation observed in existing systems is their inability to effectively integrate multi-dimensional feature representations. Phishing detection requires the analysis of various feature types, including lexical features (URL length, special characters), host-based features (domain age, DNS reputation), and structural features (URL hierarchy and depth). Traditional models often focus on a limited subset of these features, resulting in incomplete understanding of phishing behavior. A comprehensive system must be capable of combining multiple feature types to improve detection performance.

To address these challenges, there is a clear need for a hybrid and adaptive learning framework that can automatically extract meaningful features, capture complex relationships within the data, and provide robust classification results. The system must be capable of learning both local structural patterns and global sequential dependencies, while also maintaining computational efficiency for real-time deployment.

In summary, the problem addressed in this research is the design and development of an advanced phishing detection system that satisfies the **following requirements:**

- ✧ Accurate classification of URLs into legitimate and phishing categories
- ✧ Ability to detect zero-day and previously unseen phishing attacks
- ✧ Effective handling of complex and obfuscated URL structures
- ✧ Integration of multi-dimensional feature representations
- ✧ Reduction of false positive and false negative rates
- ✧ Efficient operation in real-time and large-scale environments

The proposed solution aims to fulfill these requirements by leveraging a hybrid deep learning architecture, combining CNN, BiLSTM, and XGBoost, to achieve improved detection accuracy, robustness, and scalability.

1.4 PROPOSED SOLUTION AND KEY CONTRIBUTIONS

The limitations observed in existing phishing detection systems highlight the necessity for a more advanced, adaptive, and intelligent framework capable of addressing the complexities of modern phishing attacks. To overcome these challenges, this research proposes a Hybrid Deep Learning-based Phishing Detection System, which integrates multiple learning paradigms to achieve superior detection performance, robustness, and scalability. The proposed framework combines Convolutional Neural Networks (CNN), Bidirectional Long Short-Term

Memory (BiLSTM) networks, and XGBoost classification, forming a multi-stage architecture designed to capture diverse characteristics of phishing URLs.

1.4.1 OVERVIEW OF THE PROPOSED HYBRID FRAMEWORK

To address the limitations of existing phishing detection systems, this research proposes a Hybrid Deep Learning-based Phishing Detection Framework that integrates multiple learning techniques to improve accuracy, robustness, and scalability. The proposed system is designed as a multi-stage pipeline, where each stage performs a specific function in transforming raw URL data into meaningful predictions.

Unlike traditional systems that rely on a single model, this framework combines CNN, BiLSTM, and XGBoost, enabling it to capture diverse characteristics of phishing URLs. This hybrid design ensures that both structural patterns and sequential dependencies are effectively learned, resulting in a more comprehensive and intelligent detection mechanism capable of handling modern and evolving phishing attacks.

1.4.2 DATA PREPROCESSING AND INPUT REPRESENTATION

The first stage of the proposed system focuses on data preprocessing, which is essential for ensuring the quality and consistency of input data. Raw URLs often contain noise, redundant symbols, and inconsistent formatting, which can negatively impact model performance. Therefore, preprocessing techniques such as URL normalization, tokenization, lowercase conversion, and padding are applied to standardize the data.

Each URL is transformed into a character-level numerical representation, preserving both its structural and sequential information. This representation allows the deep learning models to effectively learn patterns from the input data. By ensuring a clean and structured input format, this stage enhances the

reliability and efficiency of subsequent processing steps.

1.4.3 STRUCTURAL FEATURE EXTRACTION USING CNN

The next stage employs a Convolutional Neural Network (CNN) to extract local structural features from the input URLs. CNN is particularly effective in identifying patterns such as suspicious characters, repeated symbols, and abnormal substring arrangements, which are common indicators of phishing behavior. The convolutional filters slide across the input sequence, capturing important features at different positions.

Through multiple layers, the CNN learns hierarchical representations, where simple patterns are identified in lower layers and more complex patterns are captured in deeper layers. This enables the system to detect subtle anomalies that may not be visible through traditional methods. As a result, CNN plays a crucial role in understanding the structural characteristics of phishing URLs.

1.4.4 SEQUENTIAL PATTERN LEARNING USING BILSTM

To complement the structural analysis performed by CNN, the proposed system incorporates a Bidirectional Long Short-Term Memory (BiLSTM) network for capturing sequential dependencies within URLs. Unlike standard LSTM models, BiLSTM processes the input data in both forward and backward directions, allowing it to learn relationships between different parts of the URL more effectively.

This is particularly important because the meaning of a URL often depends on the arrangement and interaction of its components. The BiLSTM layer generates context-aware feature representations, which capture complex patterns that cannot be identified through structural analysis alone. By combining forward and backward context, the model achieves a deeper understanding of phishing behavior.

1.4.5 HYBRID DEEP FEATURE REPRESENTATION

The outputs of the CNN and BiLSTM modules are combined to form a hybrid deep feature representation, which encodes both spatial (structural) and temporal (sequential) information. This integration provides a comprehensive view of the input data, enabling the system to analyze multiple dimensions of phishing patterns simultaneously.

The combined feature vector captures detailed insights into the characteristics of URLs, improving the model's ability to distinguish between legitimate and phishing websites. This hybrid representation serves as a powerful input for the final classification stage, ensuring that the system benefits from both structural and contextual information.

1.4.6 FINAL CLASSIFICATION USING XGBOOST

Instead of relying solely on neural network outputs, the proposed system utilizes XGBoost, a highly efficient ensemble learning algorithm, for the final classification. XGBoost is based on gradient boosting decision trees, which are capable of modeling complex non-linear relationships between features. By using the deep features extracted from CNN and BiLSTM, XGBoost enhances prediction accuracy and reduces the risk of overfitting.

This approach improves the stability of the model and minimizes false positive rates. The separation of feature extraction and classification stages allows each component to perform its task more effectively, resulting in a more reliable and accurate detection system.

1.4.7 SYSTEM EFFICIENCY AND REAL-TIME DEPLOYMENT

The proposed framework is designed with a strong focus on real-time performance and scalability, making it suitable for practical deployment in cybersecurity applications. By optimizing the architecture and feature representation, the system achieves a balance between computational efficiency and detection accuracy.

This allows it to process large volumes of URL data with minimal latency. The model can be integrated into applications such as browser extensions, web security gateways, and enterprise-level monitoring systems, where rapid detection is critical. Its modular design also ensures that it can be easily extended or modified to accommodate new requirements and evolving threats.

1.4.8 KEY CONTRIBUTIONS OF THE PROPOSED SYSTEM

The proposed phishing detection framework introduces several significant contributions to the field of cybersecurity. It presents a novel hybrid architecture that combines CNN and BiLSTM to capture multiple dimensions of phishing patterns. It also introduces a deep feature-driven classification approach using XGBoost, which enhances prediction accuracy and stability.

The system demonstrates improved performance across key metrics such as accuracy, precision, recall, and F1-score, while reducing false positives and false negatives. Additionally, the framework is designed for real-time deployment, ensuring practical usability in real-world environments. Its ability to detect zero-day phishing attacks and adapt to evolving patterns makes it a robust and scalable solution for modern cybersecurity challenges.

1.5 ORGANISATION OF THE REPORT

The structure of this thesis is systematically designed to provide a clear and comprehensive understanding of the proposed phishing detection framework, starting from fundamental concepts to advanced implementation and evaluation. Each chapter is organized in a logical sequence to ensure smooth progression of ideas and to facilitate easy understanding of the research contributions.

Chapter 1 – Introduction presents the overall context of the research, including the background, motivation, and challenges associated with phishing detection. It also defines the problem statement and introduces the proposed hybrid solution, highlighting its significance in addressing the limitations of existing systems. This chapter serves as the foundation for the entire study by outlining the objectives and scope of the research.

Chapter 2 – Literature Review provides a detailed analysis of existing work in the domain of phishing detection. It covers various approaches, including traditional rule-based methods, machine learning techniques, and deep learning models. The chapter critically evaluates these methods, discussing their strengths and limitations, and identifies key research gaps that motivate the development of the proposed system.

Chapter 3 – Proposed System Architecture describes the design and structure of the proposed framework in detail. It explains how different components such as data preprocessing, CNN-based feature extraction, BiLSTM-based sequential learning, and XGBoost classification are integrated to form a unified system. This chapter provides a high-level overview of the system workflow along with architectural insights.

Chapter 4 – Methodology focuses on the technical implementation of the proposed system. It explains the complete workflow, including data preparation, feature extraction process, model training, and classification strategy. This

chapter also includes algorithmic explanations and mathematical formulations that define the working of each module in the system

Chapter 5 – Experimental Results and Evaluation presents the performance analysis of the proposed model. It describes the dataset used, evaluation metrics, and experimental setup. The results are compared with baseline models to demonstrate improvements in terms of accuracy, precision, recall, and F1-score, along with a discussion on system efficiency and scalability.

Finally, Chapter 6 – Conclusion and Future Work summarizes the key findings of the research and highlights the contributions of the proposed system. It also discusses the limitations of the current approach and suggests potential directions for future enhancements, such as extending the model to other domains like email phishing or improving real-time detection capabilities.

Overall, this structured organization ensures a comprehensive and systematic presentation of the research, enabling readers to clearly understand the problem, proposed solution, and its effectiveness in addressing modern phishing detection challenges.

CHAPTER 2

LITERATURE REVIEW

Phishing detection has been widely explored in cybersecurity due to the increasing number of malicious websites and online fraud attacks. Several research works have focused on improving detection accuracy using machine learning, deep learning, and feature selection techniques. According to the study in [2], phishing attacks have evolved from simple email scams to highly sophisticated techniques that can bypass traditional security mechanisms.

Existing approaches can be broadly categorized into heuristic-based methods, machine learning models, deep learning architectures, and hybrid frameworks. Each category contributes to phishing detection but also introduces certain limitations, which motivate the need for an improved hybrid solution.

2.1 TRADITIONAL RULE-BASED AND BLACKLIST-BASED DETECTION SYSTEMS

Traditional phishing detection systems were primarily built on rule-based mechanisms and blacklist filtering techniques, which formed the foundation of early cybersecurity solutions. These systems operate by comparing URLs against a database of known malicious websites or by applying predefined rules to identify suspicious characteristics. For instance, URLs containing excessive special characters, long domain structures, or numerical IP addresses instead of domain names are often flagged as phishing attempts. Such approaches are computationally efficient and easy to deploy, making them widely used in early web security systems.

However, despite their simplicity, these methods suffer from critical limitations in modern environments. The most significant drawback is their inability to detect zero-day phishing attacks^{xxv}, where newly generated malicious URLs are not yet present in the blacklist database. Attackers frequently create

short-lived domains and dynamically modify URL structures, making it difficult for static systems to keep up. Additionally, rule-based systems rely heavily on manually defined heuristics, which require continuous updates and expert intervention. As phishing techniques evolve, these rules become outdated, leading to reduced detection accuracy.

Another important limitation is the lack of contextual understanding in traditional systems. These methods focus only on surface-level patterns and fail to capture deeper relationships within URLs. As a result, sophisticated phishing websites that closely mimic legitimate domains can easily bypass detection. Furthermore, maintaining large-scale blacklist databases introduces scalability challenges, especially in real-time applications. These limitations clearly demonstrate that traditional approaches are insufficient for handling the complexity of modern phishing attacks.

2.2 MACHINE LEARNING-BASED PHISHING DETECTION APPROACHES

The introduction of machine learning techniques marked a significant improvement over traditional phishing detection systems by enabling models to learn patterns from data rather than relying on static rules. Algorithms such as Support Vector Machines (SVM), Random Forest, Naïve Bayes, and Logistic Regression have been widely used for classifying URLs as phishing or legitimate. These models utilize a variety of features, including lexical attributes (URL length, special characters), domain-based features (domain age, WHOIS data), and statistical properties of URLs.

Machine learning models offer improved adaptability compared to rule-based systems, as they can generalize from training data and identify patterns that are not explicitly defined. This allows them to detect previously unseen phishing URLs to some extent.

Additionally, ensemble methods such as Random Forest provide better robustness by combining multiple decision trees, thereby improving classification accuracy and reducing variance. Despite these advantages, machine learning approaches face several challenges. One of the primary limitations is their reliance on manual feature engineering, which requires domain expertise and extensive preprocessing.

Selecting relevant features is a complex task, and poor feature selection can significantly degrade model performance. Moreover, these models often struggle to capture non-linear and complex relationships present in phishing URLs, especially when attackers use advanced obfuscation techniques.

Another issue is the limited generalization capability of machine learning models when applied to different datasets. Models trained on one dataset may not perform well on another due to variations in feature distribution. Additionally, as phishing attacks become more sophisticated, attackers design URLs that closely resemble legitimate ones, making it difficult for traditional classifiers to differentiate between them. These limitations highlight the need for more advanced approaches capable of automatically learning complex patterns from raw data.

2.3 DEEP LEARNING-BASED PHISHING DETECTION MODELS

The emergence of deep learning techniques has revolutionized the field of phishing detection by enabling automatic feature extraction and improved pattern recognition. Unlike traditional machine learning models, deep learning algorithms can learn hierarchical representations directly from raw data, eliminating the need for manual feature engineering. Models such as Convolutional Neural Networks (CNN) and Recurrent Neural Networks (RNN) have been extensively used for phishing detection tasks.

CNN models are particularly effective in capturing local structural patterns within URLs. By applying convolutional filters, CNN can detect suspicious substrings, repeated characters, and abnormal symbol combinations that are commonly found in phishing URLs. These models treat URLs as sequences of characters and extract meaningful patterns through multiple convolutional layers. On the other hand, RNN-based models, especially Long Short-Term Memory (LSTM) networks, are designed to capture sequential dependencies within data. They analyze the order and relationships between characters, enabling the detection of contextual patterns that indicate phishing behavior.

While deep learning models offer significant advantages, they also have certain limitations. CNN models focus mainly on structural features and may overlook sequential dependencies, whereas LSTM models capture sequence information but may not effectively identify local structural anomalies. Additionally, deep learning models require large amounts of training data and computational resources, which can limit their practical deployment. Training these models can be time-consuming, and their complexity may introduce latency in real-time applications.

To overcome these challenges, researchers have started exploring hybrid approaches that combine different deep learning models. By integrating CNN and LSTM architectures, it is possible to leverage the strengths of both models and achieve better performance in phishing detection tasks.

2.4 HYBRID DEEP LEARNING AND URL INTELLIGENCE BASED FRAMEWORKS

To Recent advancements in phishing detection research have focused on the development of hybrid models that combine multiple learning techniques to improve detection accuracy and robustness.

One of the most effective approaches involves integrating CNN and BiLSTM, where CNN extracts structural features and BiLSTM captures sequential dependencies. This combination allows the system to analyze both spatial and temporal characteristics of URLs, providing a more comprehensive understanding of phishing patterns.

In addition to hybrid deep learning architectures, researchers have introduced URL intelligence-based frameworks, which incorporate multiple types of features, including lexical features, host-based attributes, and structural characteristics. These frameworks analyze various aspects of URLs, such as domain reputation, DNS information, and URL hierarchy, to identify phishing behavior. Advanced techniques also include feature optimization and relevance ranking, which help in selecting the most important features for classification.

Another important development is the use of ensemble learning algorithms, such as XGBoost, in combination with deep learning models. By using deep features as input, ensemble methods improve classification accuracy and reduce false positive rates. This hybrid approach enhances the stability and reliability of the system, making it more suitable for real-world applications.

However, hybrid models also introduce challenges related to computational complexity and scalability. Combining multiple models increases the overall complexity of the system, which can affect processing speed and

resource usage. Ensuring efficient implementation and real-time performance remains a key challenge in this area. Despite these challenges, hybrid frameworks represent a promising direction for improving phishing detection systems.

2.5 LIMITATIONS OF EXISTING APPROACHES

The comprehensive review of existing phishing detection approaches reveals several important research gaps that need to be addressed. Firstly, traditional and machine learning-based systems lack the ability to effectively detect zero-day phishing attacks, as they rely on static rules or manually engineered features. Secondly, single-model deep learning approaches fail to capture both structural and sequential characteristics simultaneously, leading to incomplete pattern understanding.

Another significant gap is the lack of multi-dimensional feature integration, where existing systems fail to combine lexical, structural, and host-based features effectively. This limits their ability to detect sophisticated phishing attacks that involve multiple layers of obfuscation. Additionally, many advanced models suffer from high computational complexity, making them unsuitable for real-time deployment in practical environments.

Furthermore, existing systems often exhibit high false positive and false negative rates, which reduce their reliability and usability. The lack of scalability and adaptability further limits their effectiveness in handling large-scale and evolving datasets. These challenges highlight the need for a more robust and intelligent phishing detection framework.

Motivated by these gaps, this research proposes a Hybrid CNN–BiLSTM–XGBoost framework, which combines deep learning-based feature extraction with ensemble classification. This approach aims to improve detection accuracy, reduce false predictions, and ensure scalability for real-time applications. By addressing the limitations of existing systems, the proposed model provides a comprehensive solution for modern phishing detection challenges.

Table 2.1: Comparative Analysis of Phishing Detection Methods

No.	Domain	Existing Method	Real-time / Scalable	Explainable	Efficiency
1	Phishing URL Detection	LSD Voting Model (LR+SVC+DT)	✓	✗	High
2	Malicious URL Detection	CTI Two-Stage Ensemble (RF+MLP)	✗	✗	High
3	Phishing Website Detection	Hyperlink-Based DNN (HB-DNN)	✗	✗	Medium
4	CPS Web Security	XAIAOA-WPC (MHA-LSTM + XAI)	✗	✓	Very High
5	Phishing URL Detection	DM-PSO-PUD	✓	✗	Medium
6	Phishing Classification	FT-MLPD (Tuned ML Models)	✓	✗	High
7	Phishing URL Detection	PhiUSIIL (Similarity + Incremental)	✓	✗	Very High
8	URL Defacement Detection	PSO-FS-UDD	✓	✗	High
9	Phishing Website Detection	PSO-FOPDF	✗	✗	High
10	Phishing Detection	HawkPhish-DNN	✓	✗	Very High
11	Phishing URL Prediction	IWQPSO-FMRCNN	✗	✗	High

13	Phishing URL Detection	RP-PUD (Residual Pipeline + MLP)	✓	✗	High
14	Phishing Classification	NI-SVM-OM (Nature-Inspired SVM)	✗	✗	Medium
15	IoT Phishing Detection	GA-OEPD (GA-Optimized Ensemble)	✓	✗	High

CHAPTER 3

PROPOSED SYSTEM ARCHITECTURE

The increasing complexity of phishing attacks necessitates the development of an intelligent and scalable detection framework capable of capturing both structural and contextual patterns present in malicious URLs. In this research, a Hybrid Deep Learning-based Phishing Detection Architecture is proposed by integrating Convolutional Neural Networks (CNN), Bidirectional Long Short-Term Memory (BiLSTM) networks, and an XGBoost classification module within a unified system.

The architecture is designed to process raw URL inputs, transform them into meaningful feature representations, and generate accurate predictions with minimal computational overhead. Unlike conventional systems that rely on static rules or isolated models, the proposed architecture adopts a multi-stage hierarchical processing pipeline, where each stage contributes to progressive feature extraction, representation learning, and classification refinement.

The overall system is structured to ensure that both local structural anomalies and long-range sequential dependencies within URLs are effectively captured. CNN is responsible for identifying localized patterns such as suspicious substrings and irregular character sequences, while BiLSTM captures contextual relationships by analyzing the sequence in both forward and backward directions.

The extracted deep features are further processed using XGBoost, an ensemble learning algorithm that enhances classification accuracy by modeling complex non-linear relationships. This combination ensures that the system is capable of detecting both known phishing URLs and previously unseen attack patterns, thereby improving robustness and generalization.

3.1 ARCHITECTURAL DESIGN PRINCIPLES

The proposed system architecture is guided by a set of carefully defined design principles that ensure robustness, scalability, and adaptability in phishing detection. One of the fundamental principles is the concept of multi-dimensional feature learning, where the system is designed to capture multiple aspects of URL behavior, including lexical composition, structural arrangement, and sequential dependencies. By combining CNN and BiLSTM, the architecture ensures that both spatial and temporal features are effectively learned, overcoming the limitations of single-model approaches.

Another important principle is progressive representation learning, where the system transforms raw input data into increasingly abstract feature representations through multiple processing stages. Instead of directly performing classification on raw URLs, the architecture first extracts low-level features, then refines them into higher-level representations, and finally uses these refined features for classification. This layered approach improves both detection accuracy and computational efficiency.

The architecture also emphasizes the decoupling of feature extraction and classification, allowing each component to specialize in its respective function. Deep learning models focus on extracting meaningful features, while the XGBoost classifier performs decision-making based on these features. This separation improves model stability, reduces overfitting, and enhances interpretability.

Additionally, the system is designed with a focus on real-time scalability, ensuring that it can process large volumes of URL data without significant latency. The modular design allows for easy updates and extensions, making the system adaptable to evolving phishing techniques.

3.2 HIGH-LEVEL SYSTEM OVERVIEW

The proposed phishing detection system follows a structured pipeline architecture consisting of multiple interconnected modules that work sequentially to process input data and generate predictions. At a high level, the system accepts raw URLs as input and passes them through stages of preprocessing, feature extraction, sequential modeling, and classification.

The preprocessing module prepares the input data by normalizing and encoding URLs into numerical sequences. These sequences are then passed to the CNN module, which extracts structural patterns using convolutional filters. The output of the CNN is further processed by the BiLSTM module, which captures contextual dependencies within the sequence. The combined output forms a comprehensive feature representation that is fed into the XGBoost classifier for final prediction.

This pipeline ensures that each stage contributes to refining the input data, resulting in improved detection accuracy and robustness. In addition to its structured workflow, the proposed system emphasizes the importance of feature quality and representation learning, which play a crucial role in improving classification performance. By transforming raw URL data into a meaningful numerical format, the system enables deep learning models to identify hidden patterns that are not easily detectable through traditional methods.

The integration of CNN and BiLSTM ensures that both local structural characteristics and long-range sequential dependencies are effectively captured, providing a more comprehensive understanding of phishing behavior. Furthermore, the use of XGBoost as the final classifier enhances prediction reliability by modeling complex relationships between extracted features. This combination of deep learning and ensemble learning not only improves accuracy but also reduces false predictions, making the system more robust and suitable for deployment in dynamic and large-scale cybersecurity environments.

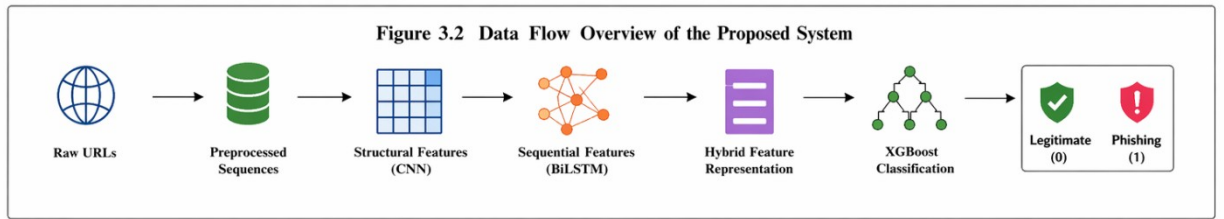
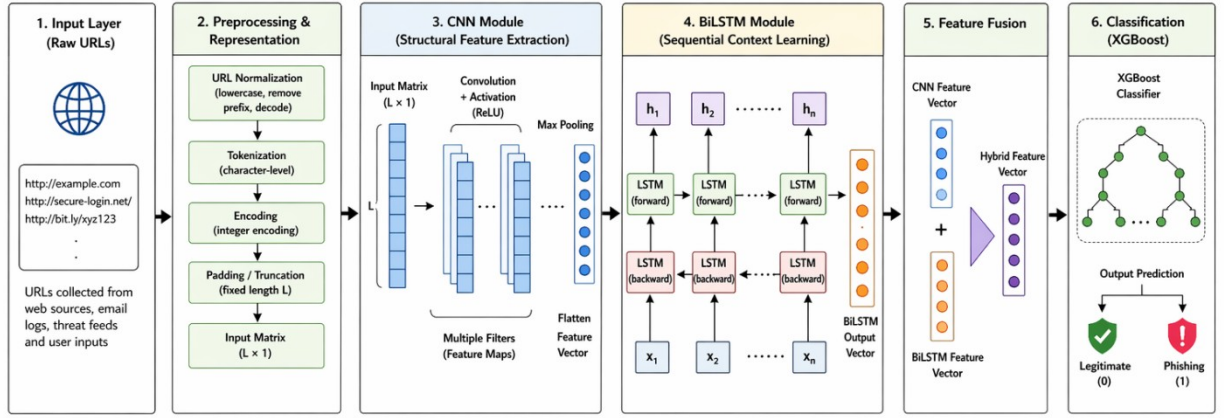


Figure 3.1: Overall architecture of the proposed CNN-BiLSTM-XGBOOST

Powered Deep Learning Phishing Detection

Figure 3.1 illustrates the overall architecture of the proposed phishing detection system. The diagram begins with the input layer, where raw URLs are provided to the system. These URLs are first processed by the preprocessing module, which converts them into structured numerical sequences. The processed data is then passed to the CNN module, where convolutional operations extract local structural patterns.

The output of the CNN is fed into the BiLSTM module, which processes the sequence in both forward and backward directions to capture contextual relationships. This step ensures that the system understands the dependencies between different parts of the URL. The outputs of the CNN and BiLSTM modules are combined to form a hybrid feature vector, which represents both structural and sequential characteristics.

Finally, the hybrid feature vector is passed to the XGBoost classifier, which performs the final classification by analyzing the relationships between features. The output of the system is a binary decision indicating whether the URL is legitimate or phishing. The diagram highlights the flow of data through each stage of the system and demonstrates how different components interact to achieve accurate detection.

3.3 URL PREPROCESSING AND REPRESENTATION LAYER

The preprocessing stage plays a critical role in ensuring that the input data is suitable for deep learning models. Raw URLs often contain inconsistencies, encoded characters, and noise that can negatively impact model performance. Therefore, a series of preprocessing steps are applied to standardize the data and improve its quality.

The process begins with normalization, where URLs are converted to lowercase and unnecessary prefixes are removed. Encoded characters are decoded to preserve the original structure of the URL. The URLs are then tokenized at the character level, allowing the system to capture fine-grained patterns within the data. These tokens are converted into numerical representations using encoding techniques, and padding is applied to ensure uniform sequence length.

This representation preserves both the structural and sequential characteristics of URLs, enabling the deep learning models to effectively learn patterns associated with phishing behavior.

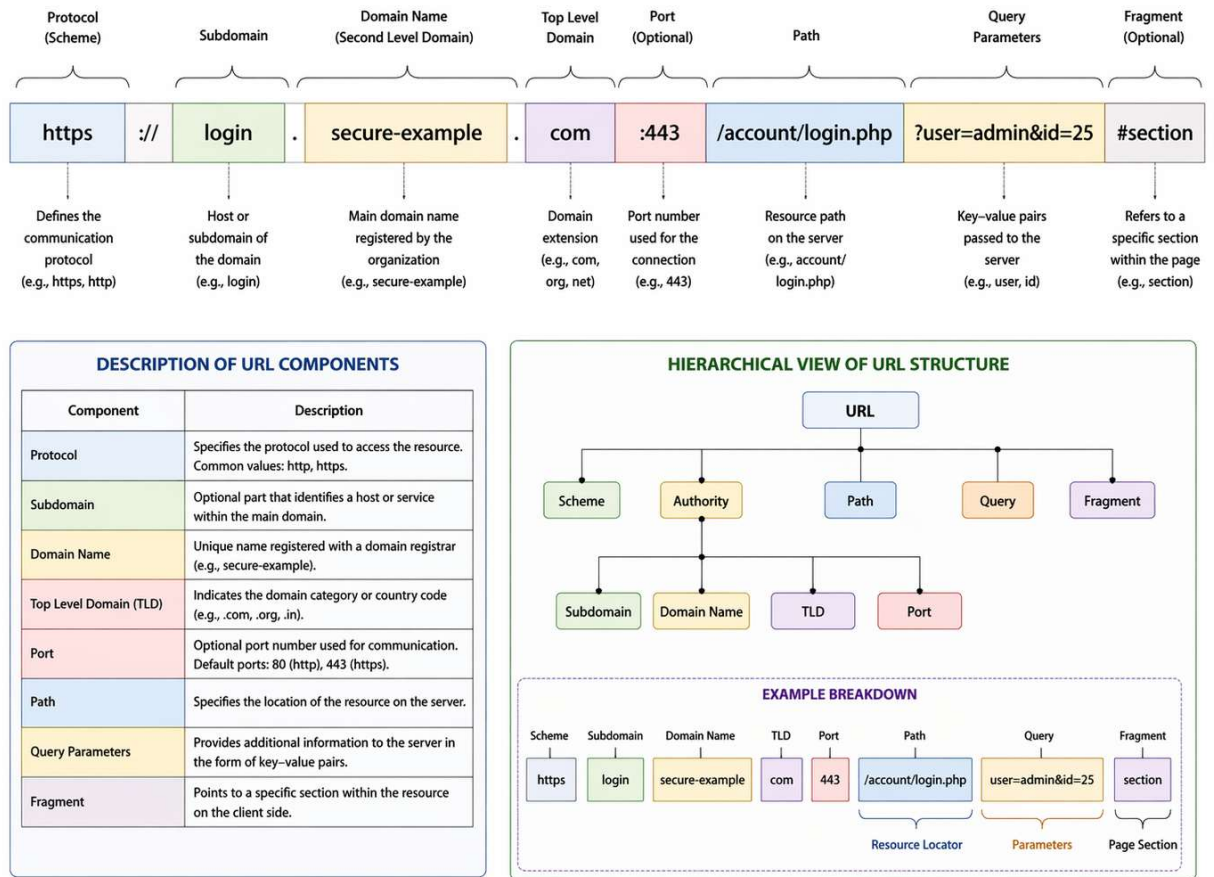


Figure 3.2: Structure of a URL and Its Components

3.4 CNN-BASED STRUCTURAL FEATURE EXTRACTION

The CNN module is responsible for extracting structural patterns from the input data. By applying convolutional filters, the model scans the input sequence and identifies patterns such as suspicious characters, repeated symbols, and abnormal substring arrangements. These patterns are often indicative of phishing behavior and provide valuable insights into the structure of malicious URLs.

Through multiple convolutional layers, the CNN learns hierarchical representations of the input data. Lower layers capture simple patterns, while deeper layers identify more complex structures. This hierarchical learning process enables the model to detect subtle anomalies that may not be visible through traditional methods.

3.5 BiLSTM-BASED SEQUENTIAL LEARNING

While CNN focuses on structural patterns, the BiLSTM module captures the sequential relationships within the URL. By processing the input sequence in both forward and backward directions, BiLSTM learns how different parts of the URL interact with each other. This is particularly important for identifying patterns where the meaning depends on the arrangement of tokens.

The output of the BiLSTM module is a context-aware representation of the input data, which provides deeper insights into phishing behavior. This representation complements the structural features extracted by the CNN, resulting in a more comprehensive understanding of the data.

3.6 HYBRID FEATURE FUSION AND REPRESENTATION

The outputs of the CNN and BiLSTM modules are combined to form a hybrid feature representation that integrates both structural and sequential information. This combined feature vector provides a comprehensive view of the input data, enabling the system to analyze multiple dimensions of phishing patterns simultaneously.

The fusion process enhances the discriminative power of the model by leveraging complementary information from different sources. This results in improved detection accuracy and robustness.

3.7 XGBOOST-BASED FINAL CLASSIFICATION

The final stage of the system involves classification using XGBoost, which analyzes the hybrid feature vector to generate predictions. XGBoost uses gradient boosting techniques to model complex relationships between features and improve classification accuracy.

By leveraging the strengths of ensemble learning, XGBoost enhances the stability and reliability of the model. It also reduces overfitting and improves generalization, making the system suitable for real-world deployment.

3.8 ARCHITECTURAL ADVANTAGES

The proposed architecture offers several advantages over existing phishing detection systems. By combining CNN and BiLSTM, the system captures both structural and sequential patterns, enabling more accurate detection of phishing URLs. The use of XGBoost further enhances classification performance by modeling non-linear relationships.

The modular design ensures scalability and allows the system to handle large volumes of data efficiently. Additionally, the architecture is capable of detecting zero-day phishing attacks and adapting to evolving threat patterns, making it a robust and reliable solution for modern cybersecurity challenges.

CHAPTER 4

METHODOLOGY

The methodology of the proposed phishing detection system defines the structured approach used to transform raw URL data into accurate classification outputs. In this research, a hybrid framework combining **CNN, BiLSTM, and XGBoost** is utilized to capture both structural and sequential characteristics of URLs. Unlike traditional approaches that rely on handcrafted features.

The proposed methodology focuses on automatic feature extraction, enabling the system to learn complex phishing patterns directly from data. Each stage in the methodology contributes to refining the input representation, improving feature quality, and enhancing classification accuracy. The overall workflow ensures that the system is capable of detecting both known and zero-day phishing attacks with high reliability.

4.1 URL DATA COLLECTION STRATEGY

The dataset is collected from multiple sources including phishing repositories and legitimate domain datasets to ensure diversity and real-world applicability. This multi-source data collection approach helps the model learn different phishing patterns and improves generalization. The dataset consists of both phishing and legitimate URLs, ensuring balanced training. By incorporating data from different time periods and sources, the system becomes capable of identifying evolving phishing techniques and detecting unseen attacks effectively.

From a mathematical perspective, the dataset can be represented as:

$$\mathbf{D} = \{(\mathbf{U}_i, \mathbf{y}_i)\}_{i=1}^N$$

This formulation allows the problem to be treated as a supervised classification task, where the objective is to learn a mapping function that can accurately predict the label for unseen URLs. The dataset preparation process involves several steps, including data cleaning, duplicate removal, and class balancing.

These steps ensure that the dataset is free from redundancy and bias, which can negatively impact model performance. Additionally, splitting the dataset into training, validation, and testing subsets ensures that the model is evaluated fairly and does not overfit to the training data. This structured approach to data collection and preparation forms the foundation for effective model training.

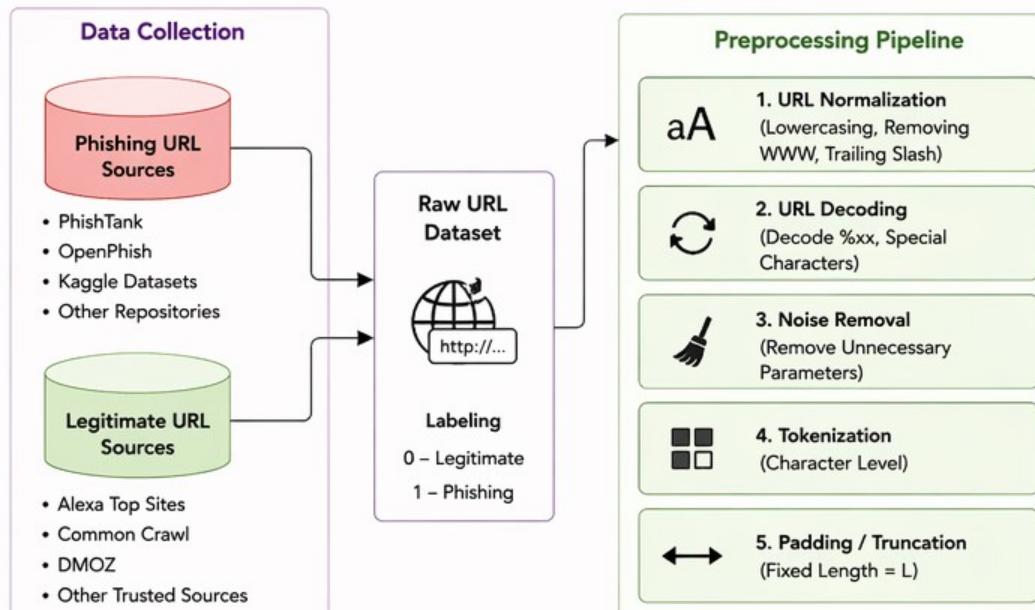


Figure 4.1: Collection of Data From Various Sources

4.2 URL DATA PREPROCESSING, REPRESENTATION AND MATHEMATICAL FORMULATION

The preprocessing stage is one of the most important components of the proposed system, as it transforms raw URL data into a structured numerical

format suitable for deep learning models. In real-world scenarios, URLs often contain noise, encoded characters, and redundant parameters that can interfere with the learning process. Therefore, preprocessing is essential to clean and standardize the data before feature extraction.

A URL is first represented as a sequence of characters:

$$\mathbf{U} = \{c_1, c_2, c_3, \dots, c_n\}$$

This representation allows the model to analyze the URL at a fine-grained level, capturing subtle character-level manipulations that are commonly used in phishing attacks. example, attackers may replace characters with visually similar alternatives or insert additional symbols to mimic legitimate domains. By treating the URL as a sequence, the model can detect these small but significant changes.

The normalization process is defined as:

$$\mathbf{U}_{\text{norm}} = \phi(\mathbf{U})$$

This function standardizes the URL by converting all characters to lowercase, removing unnecessary symbols, and eliminating redundant parameters. Normalization ensures that similar URLs are represented consistently, which improves the stability and accuracy of the model.

Encoded characters are decoded using the following transformation:

$$c_i' = \text{decode}(c_i)$$

This step converts encoded values such as “%2F” into their original characters, revealing hidden patterns that may indicate phishing behavior. Without decoding, these patterns would remain obscured, reducing the effectiveness of the detection system.

Each character is then converted into a numerical representation:

$$x_i = E(c_i)$$

This encoding process maps characters to numerical values, enabling the neural network to perform mathematical operations. The encoded sequence is represented as:

$$X = \{x_1, x_2, x_3, \dots, x_n\}$$

Finally, padding is applied to ensure uniform sequence length. This step ensures that all input sequences have the same length, which is necessary for efficient training and batch processing. Padding also helps maintain consistency across the dataset, allowing the model to process multiple inputs simultaneously.

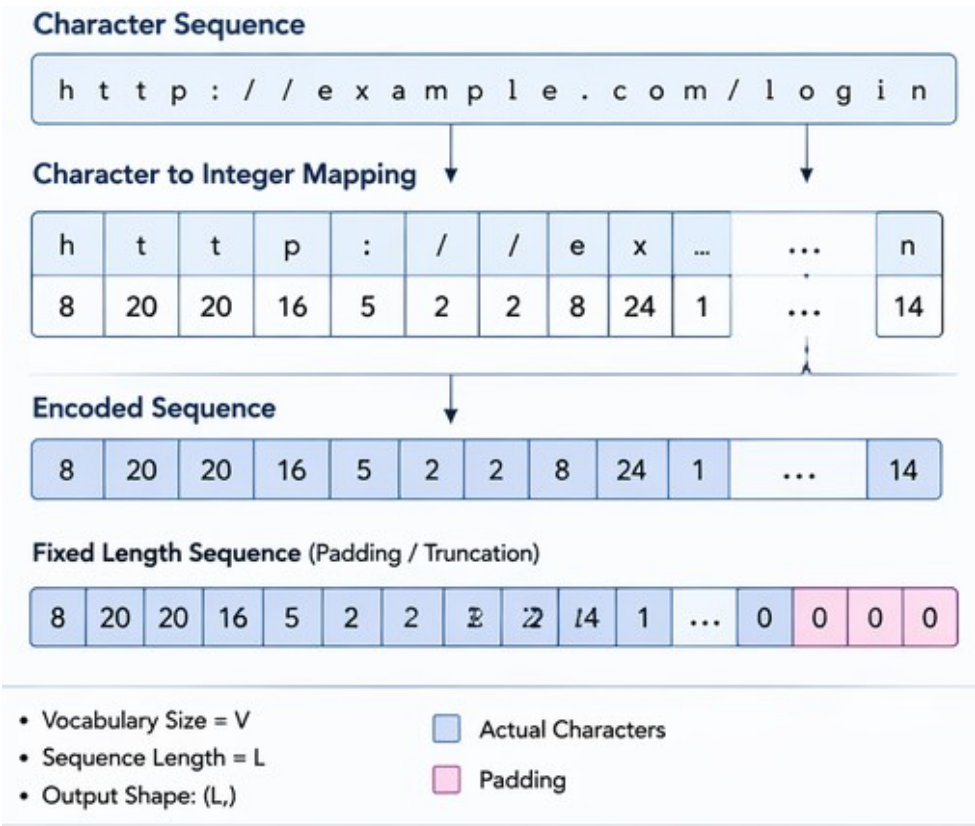


Figure 4.2: Feature Encoding And Sequence Representation

4.3 CNN-BASED STRUCTURAL FEATURE EXTRACTION

The Convolutional Neural Network (CNN) module plays a crucial role in extracting local structural patterns from the encoded URL sequences. In phishing detection, many malicious URLs rely on subtle structural manipulations such as suspicious substrings, repeated characters, and abnormal domain patterns. Traditional methods often fail to capture these fine-grained features, but CNN is specifically designed to identify such localized patterns through convolution operations. By applying multiple filters across the input sequence, the CNN automatically learns which patterns are most relevant for distinguishing between legitimate and phishing URLs. This stage transforms the numerical sequence into a rich feature representation that highlights important structural characteristics.

$$h_i = f(W \cdot X_{i:i+k-1} + b)$$

This equation defines the convolution operation, where a filter W is applied to a segment of the input sequence $X_{i:i+k-1}$. The filter slides across the sequence, capturing patterns within a window of size k . Each filter learns to detect a specific type of pattern, such as the presence of suspicious keywords or unusual character combinations. The bias term b adjusts the output, while the activation function f introduces non-linearity. This operation is essential because it allows the model to focus on local patterns rather than the entire sequence at once. In phishing detection, this is particularly useful because malicious patterns often appear in small segments of the URL.

The convolution process can be interpreted as a pattern-matching mechanism, where the filter weights are learned during training. Initially, the filters are random, but as training progresses, they adapt to detect meaningful patterns. For example, one filter may learn to identify patterns like “login-secure” or “verify-account,” while another may detect unusual sequences of numbers and

symbols. This ability to learn multiple patterns simultaneously makes CNN highly effective for feature extraction.

$$\mathbf{f}(\mathbf{x}) = \max(\mathbf{0}, \mathbf{x})$$

The ReLU activation function introduces non-linearity into the model, allowing it to capture complex relationships between input features. Without this non-linear transformation, the model would behave like a linear function and would not be able to learn intricate patterns. ReLU also improves computational efficiency and helps mitigate the vanishing gradient problem, which is common in deep neural networks. By setting negative values to zero, ReLU ensures that only meaningful features are propagated to the next layer.

$$\mathbf{H} = \{\mathbf{h1}, \mathbf{h2}, \mathbf{h3}, \dots, \mathbf{hn}\}$$

This represents the feature map generated after applying convolution across the entire sequence. Each element in the feature map corresponds to a detected pattern at a specific position in the URL. The feature map provides a detailed representation of the structural characteristics of the input. By using multiple filters, the CNN generates multiple feature maps, each capturing different aspects of the data. These maps collectively form a comprehensive representation of the URL structure.

$$\mathbf{p_j} = \max(\mathbf{h_j}, \mathbf{h_j + 1}, \dots, \mathbf{h_j + s})$$

The pooling operation reduces the dimensionality of the feature map by selecting the maximum value within a window. This step ensures that only the most important features are retained, while less significant information is discarded. Pooling also improves computational efficiency and helps prevent overfitting by reducing the number of parameters. In the context of phishing

detection, pooling ensures that the model focuses on the most critical patterns that indicate malicious behavior.

$$\mathbf{Fcnn} = \text{flatten}(\mathbf{P})$$

The flattening operation converts the pooled feature map into a one-dimensional vector, which serves as the final output of the CNN module. This vector contains the extracted structural features of the URL and is used as input for the next stage of the model. By transforming the feature map into a vector, the model can easily integrate it with other components, such as BiLSTM, for further processing.

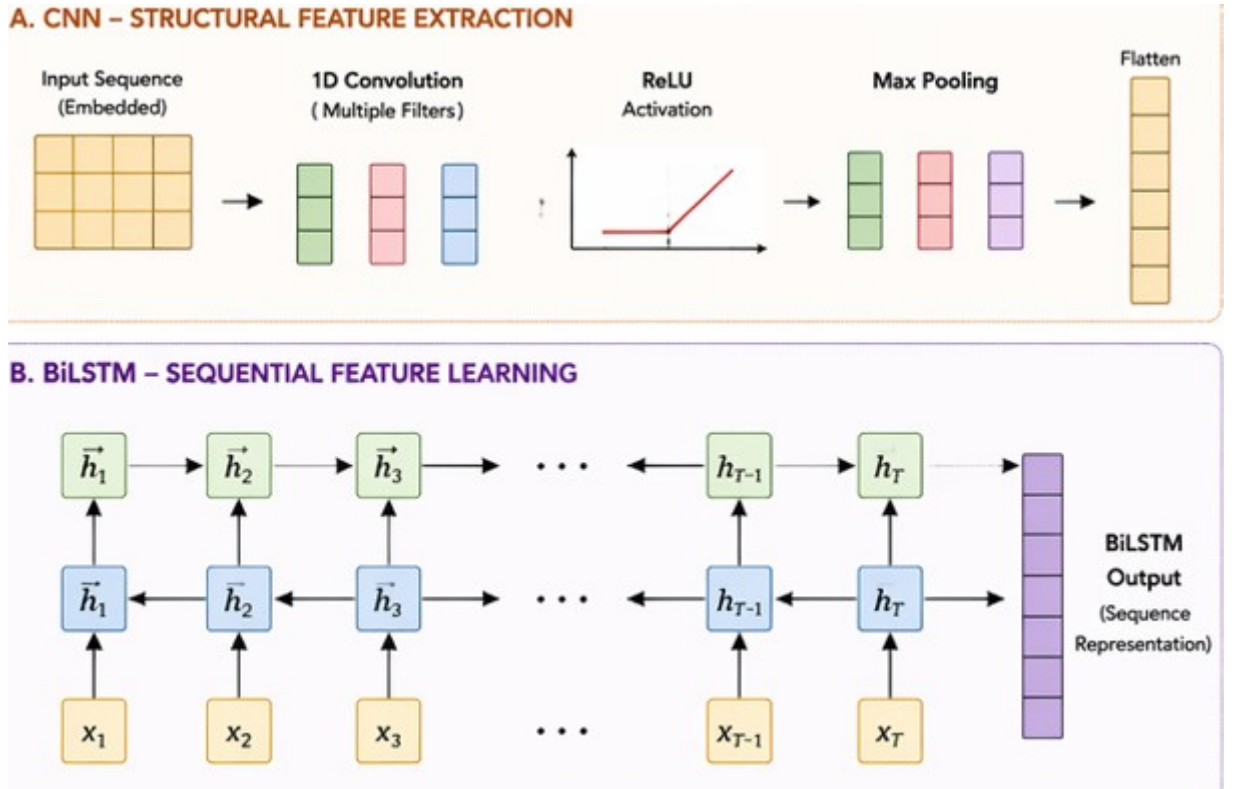


Figure 4.3: Combination of CNN + BiLSTM Extraction

4.4 BILSTM-BASED SEQUENTIAL LEARNING

The Bidirectional Long Short-Term Memory (BiLSTM) module is incorporated into the proposed system to capture sequential dependencies and contextual relationships within URL data. While CNN effectively extracts local structural patterns, it does not fully consider the order and long-range dependencies between characters. In phishing detection, the arrangement of characters plays a crucial role, as attackers often manipulate the sequence of substrings to imitate legitimate URLs. Therefore, BiLSTM is used to analyze the sequence in both forward and backward directions, enabling the model to understand the complete context of the URL.

$$\mathbf{f}_t = \sigma(\mathbf{W}_f [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f)$$

The forget gate determines which information from the previous hidden state should be discarded. This is essential because not all parts of a URL are useful for classification. For instance, common prefixes such as “http” or “www” do not contribute significantly to phishing detection. The forget gate filters out such irrelevant information, ensuring that only meaningful patterns are retained. The sigmoid function σ outputs values between 0 and 1, acting as a control mechanism that decides the importance of each piece of information.

$$\mathbf{i}_t = \sigma(\mathbf{W}_i [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i)$$

The input gate controls how much new information from the current input should be added to the memory. This allows the model to selectively incorporate features that are relevant to phishing detection. For example, if a suspicious substring appears in the URL, the input gate ensures that this information is stored in the memory cell. This selective learning process helps the model focus on important features while ignoring noise.

$$\mathbf{C}_{\sim t} = \tanh(\mathbf{W}_c [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_c)$$

The candidate memory represents new information that can be added to the cell state. The hyperbolic tangent function ensures that the values are scaled between -1 and 1, allowing both positive and negative contributions. This step prepares the new information before it is combined with the existing memory.

$$\mathbf{C}_t = \mathbf{f}_t \cdot \mathbf{C}_{t-1} + \mathbf{i}_t \cdot \mathbf{C}_{\sim t}$$

The cell state update equation combines past memory with new information. The term $\mathbf{f}_t$ retains important information from the previous state, while it adds new relevant information. This balance allows the model to capture long-term dependencies, which is critical for analyzing complex URL structures. For example, the beginning and end of a URL may both contribute to identifying phishing behavior, and this equation ensures that such relationships are preserved.

$$\mathbf{o}_t = \sigma(\mathbf{W}_o [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o)$$

The output gate determines how much of the cell state should be passed to the next layer. This ensures that only relevant information is propagated forward, improving the efficiency of the model.

$$\mathbf{h}_t = \mathbf{o}_t \cdot \tanh(\mathbf{C}_t)$$

The hidden state represents the output of the LSTM unit at each time step. It contains both past and present information, making it a powerful representation of the sequence.

$$\mathbf{h}_{tbi} = [\mathbf{h}_t, \mathbf{h}_t]$$

In BiLSTM, the sequence is processed in both forward and backward directions. This allows the model to consider both past and future context simultaneously, which is particularly useful for detecting phishing patterns that may appear anywhere in the URL.

$$\mathbf{F}_{\text{bilstm}} = \mathbf{t} = \mathbf{1} \sum^T \mathbf{h}_{\text{tbi}}$$

The final feature vector is obtained by aggregating the hidden states across all time steps. This vector captures the overall contextual information of the URL, enabling the model to understand complex relationships between characters.

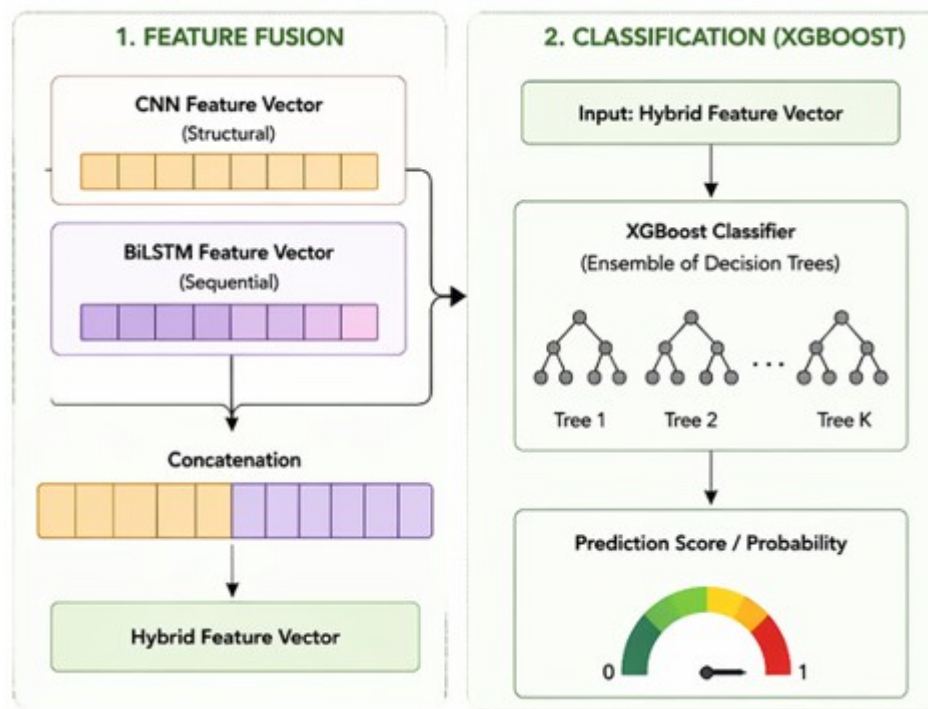


Figure 4.4: Ensemble Classification with Feature Fusion

4.5 HYBRID CNN-BILSTM-XGBOOST MODEL

The proposed hybrid model integrates three powerful components—Convolutional Neural Network (CNN), Bidirectional Long Short-Term Memory (BiLSTM), and XGBoost—to achieve highly accurate phishing detection. The

main idea behind this hybrid approach is to combine the strengths of different models so that their individual limitations are overcome, resulting in a more robust and reliable system.

At the initial stage, the CNN module processes the encoded URL sequence and extracts structural features. These features include patterns such as suspicious keywords, unusual character combinations, repeated symbols, and abnormal domain structures. CNN is particularly effective in identifying such local patterns because it focuses on small segments of the input at a time. As a result, it captures fine-grained details that are often missed by traditional methods.

Following this, the output from the CNN is passed to the BiLSTM module, which focuses on capturing sequential and contextual relationships within the URL. Unlike CNN, which looks at local patterns, BiLSTM analyzes the entire sequence and understands how different parts of the URL are related. It processes the sequence in both forward and backward directions, allowing it to capture context from both ends. This is especially useful in phishing detection, where the meaning of a URL can depend on the arrangement of characters and substrings.

After feature extraction, the outputs from CNN and BiLSTM are combined to form a hybrid feature representation:

$$\mathbf{F_h} = [\mathbf{F_{cnn}}, \mathbf{F_{bilstm}}]$$

The zero-shot formulation eliminates the requirement for task-specific annotated training data, enabling the NLI module to operate across diverse job domains without retraining. This semantic gating step ensures that candidates advancing to the final ranking stage exhibit genuine semantic alignment with the

job requirements, rather than merely superficial lexical overlap.

This step merges structural and sequential features into a single unified vector. The importance of this fusion lies in the fact that phishing URLs often contain both types of patterns. Structural features alone may not be sufficient, and sequential context alone may also miss certain anomalies. By combining both, the model gains a more complete understanding of the data.

The combined feature vector is then passed to the XGBoost classifier, which performs the final prediction. XGBoost is an ensemble learning method that builds multiple decision trees and combines their outputs to make a final decision. Instead of relying on a single model, it aggregates multiple weak learners to produce a strong classifier. This significantly improves prediction accuracy and reduces errors.

$$\mathbf{y}^{\wedge} = \mathbf{XGBoost}(\mathbf{Fh})$$

The combined feature vector is then passed to the XGBoost classifier, which performs the final prediction. XGBoost is an ensemble learning method that builds multiple decision trees and combines their outputs to make a final decision. Instead of relying on a single model, it aggregates multiple weak learners to produce a strong classifier. This significantly improves prediction accuracy and reduces errors.

The key advantage of using XGBoost is its ability to handle complex feature interactions and provide high performance even with structured data. It also includes built-in mechanisms to prevent overfitting, ensuring that the model performs well on unseen data.

During training, the model continuously learns by minimizing prediction errors and adjusting its internal parameters. XGBoost further refines the

predictions by focusing on correcting the mistakes made by previous decision trees. This iterative improvement process enhances the overall performance of the system.



Figure 4.4: Final Output Classification

In summary, the hybrid model works in a three-stage learning process:

- ✧ CNN extracts structural patterns
- ✧ BiLSTM captures sequential dependencies
- ✧ XGBoost performs final classification

This combination ensures that the system captures both local and global patterns, making it highly effective in detecting phishing URLs. Compared to individual models, the hybrid approach provides better accuracy, improved generalization, and stronger resistance to evolving phishing techniques.

4.6 PROPOSED PHISHING DETECTION SYSTEM

The methodology of the proposed phishing detection system defines the structured approach used to transform raw URL data into accurate classification outputs. In this research, a hybrid framework combining **CNN, BiLSTM, and XGBoost** is utilized to capture both structural and sequential characteristics of URLs. Unlike traditional approaches that rely on handcrafted features.

4.6.1 DATA COLLECTION

The first stage of the proposed system is data collection, which forms the foundation for model training and evaluation. In this stage, a large number of URLs are gathered from multiple reliable sources to ensure diversity and real-world applicability. Phishing URLs are collected from publicly available phishing repositories and threat intelligence platforms, where continuously updated lists of malicious URLs are maintained. Legitimate URLs are collected from trusted websites, search engine indexes, and commonly used domains.

The purpose of using multiple sources is to ensure that the dataset contains a wide variety of URL patterns, including different domain structures, lengths, and formats. This diversity is important because phishing attacks are constantly evolving, and the model must be able to generalize across different types of attacks. The collected dataset is then labeled, where each URL is assigned a class label indicating whether it is phishing or legitimate. Duplicate entries and irrelevant data are removed to improve data quality. Additionally, class balancing is performed to ensure that the dataset does not become biased toward one class, which could affect model performance. This stage ensures that the model is trained on a high-quality dataset that reflects real-world scenarios.

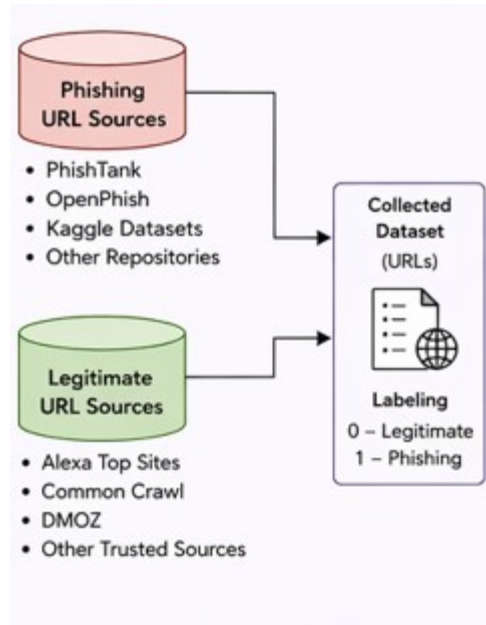


Figure 4.5: Collection of Data From Various Sources

4.6.2 DATA PREPROCESSING

The collected URLs are often noisy and inconsistent, making preprocessing a crucial step in the methodology. In this stage, the raw URLs are cleaned and standardized to ensure that they are suitable for further processing. The preprocessing pipeline performs several operations, including normalization, decoding, and noise removal.

Normalization involves converting all characters in the URL to lowercase and removing unnecessary variations such as duplicate slashes, trailing symbols, or irrelevant query parameters. This ensures consistency across the dataset. Decoding is performed to convert encoded characters into their original form, as attackers often use encoding techniques to hide malicious content. For example, encoded symbols and special characters are transformed back into readable format so that the model can analyze them effectively.

Noise removal is another important step, where unnecessary components

such as session IDs, tracking parameters, and redundant substrings are eliminated. These elements do not contribute to phishing detection and may introduce unwanted complexity. After cleaning, the URLs are tokenized into sequences of characters, which allows the model to analyze the data at a fine-grained level. This preprocessing stage ensures that the input data is clean, consistent, and meaningful, which significantly improves the performance of the model.

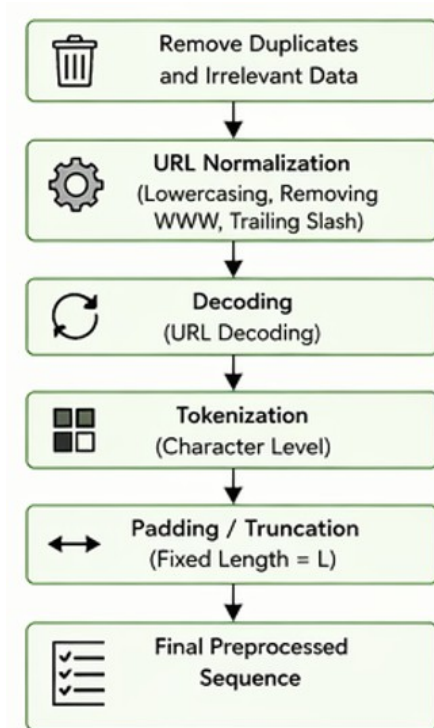


Figure 4.6: Pre-Processing the Collected Data

4.6.3 FEATURE ENCODING AND SEQUENCE TRANSFORMATION

After preprocessing, the textual URL data is converted into a numerical format that can be processed by deep learning models. Each character in the URL is mapped to a numerical value using an encoding technique. This conversion preserves the order and structure of the URL, which is essential for detecting phishing patterns.

Once encoded, the URL is represented as a sequence of numbers. Since URLs can have different lengths, the sequences are standardized to a fixed length

using padding. Shorter URLs are padded with zeros, while longer URLs are truncated to fit the required size. This ensures that all input sequences have the same dimensions, allowing efficient batch processing during training.

This stage also ensures that both structural and sequential information is retained in the encoded data. By maintaining the order of characters, the model can analyze how different parts of the URL relate to each other. This transformation serves as the input for the deep learning modules that follow.

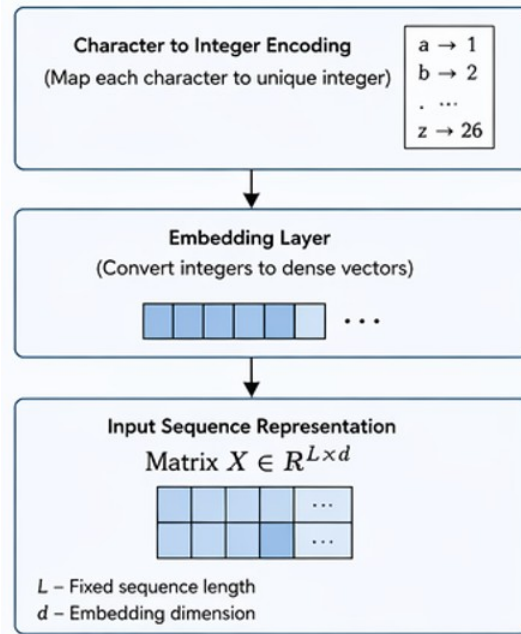


Figure 4.7: Represented the Encoded URLs

4.6.4 CNN-BASED STRUCTURAL FEATURE EXTRACTION

The encoded URL sequences are passed to the Convolutional Neural Network (CNN) module, which is responsible for extracting structural features. The CNN uses multiple filters that slide across the input sequence to detect local patterns such as suspicious substrings, unusual character combinations, and abnormal domain structures.

Each filter learns to identify a specific type of pattern, and multiple filters are used to capture different features simultaneously. The CNN focuses on local

regions of the URL, making it effective at identifying small but important patterns that may indicate phishing behavior. After convolution, activation functions are applied to introduce non-linearity, allowing the model to learn complex relationships.

Pooling operations are then used to reduce the dimensionality of the data while preserving the most important features. This helps in improving computational efficiency and prevents overfitting. The final output of the CNN is a feature vector that represents the structural characteristics of the URL. This vector highlights patterns that are commonly associated with phishing attacks.

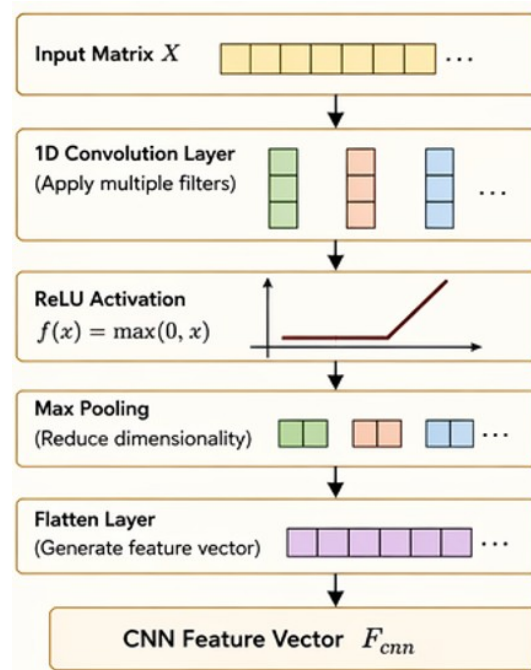


Figure 4.8: Structural Feature Extraction

4.6.5 BILSTM-BASED SEQUENTIAL LEARNING

The output of the CNN module is passed to the Bidirectional Long Short-Term Memory (BiLSTM) module, which captures sequential dependencies and contextual relationships within the URL. While CNN focuses on local patterns, BiLSTM analyzes the entire sequence to understand how different parts of the URL are connected.

BiLSTM processes the sequence in both forward and backward directions, allowing it to capture context from both ends. This is particularly useful in phishing detection, where the meaning of a URL may depend on the arrangement of characters. For example, certain patterns may appear at the beginning or end of the URL, and BiLSTM ensures that these relationships are not missed..

The BiLSTM module maintains an internal memory that helps it retain important information over long sequences. It selectively updates this memory by retaining relevant patterns and discarding irrelevant ones. This results in a context-aware feature representation that complements the structural features extracted by the CNN.

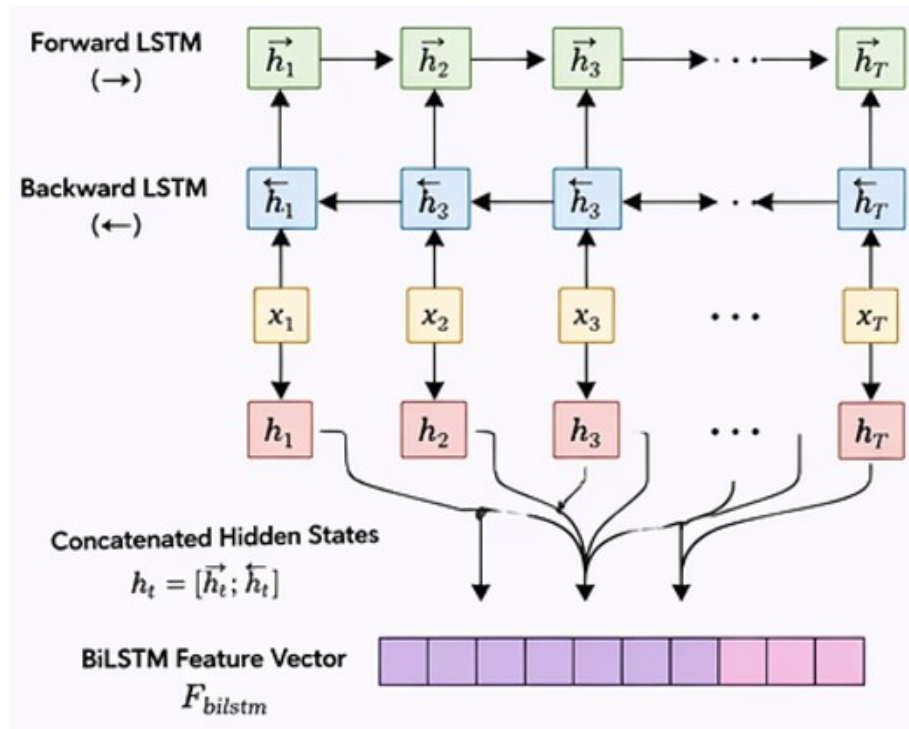


Figure 4.9: Bi-LSTM Sequential Learning

4.6.6 FEATURE FUSION

After extracting features from both CNN and BiLSTM, the system

combines these features into a single representation. This process is known as feature fusion. The structural features from CNN and the sequential features from BiLSTM are merged to create a comprehensive feature vector.

This combined representation provides a more complete understanding of the URL, as it includes both local and global information. Feature fusion significantly improves the model's ability to detect phishing patterns, as it leverages the strengths of both modules. This step ensures that no important information is lost and that the model has access to all relevant features for classification.

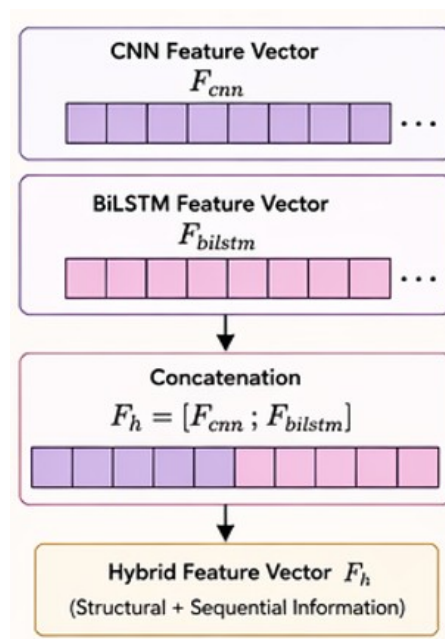


Figure 4.10: Feature Fusion

4.6.7 XGBOOST-BASED CLASSIFICATION

The fused feature vector is passed to the XGBoost classifier, which performs the final classification. XGBoost is an ensemble learning algorithm that builds multiple decision trees and combines their outputs to make predictions. It is highly effective in handling complex patterns and improves classification accuracy.

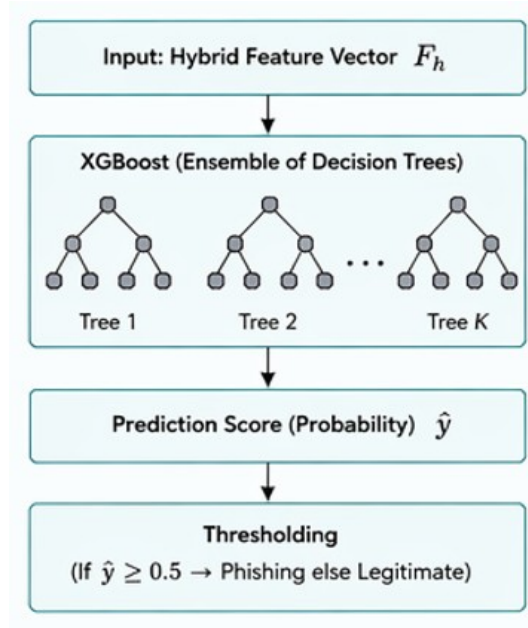


Figure 4.11: XGBoost Classifier

The classifier analyzes the input features and determines whether the URL is phishing or legitimate. By using multiple trees, XGBoost reduces errors and improves robustness. It also includes regularization techniques that prevent overfitting, ensuring that the model performs well on unseen data.

4.6.8 OUTPUT PREDICTION AND EVALUATION

The final stage of the system generates the output prediction, indicating whether the input URL is phishing or legitimate. Along with the prediction, performance metrics such as accuracy, precision, recall, and F1-score are calculated to evaluate the effectiveness of the model.

This stage also helps in analyzing the strengths and weaknesses of the system. By evaluating the results, improvements can be made to further enhance performance. The final output can be integrated into real-world applications such as browser extensions, email filtering systems, and cybersecurity platforms.

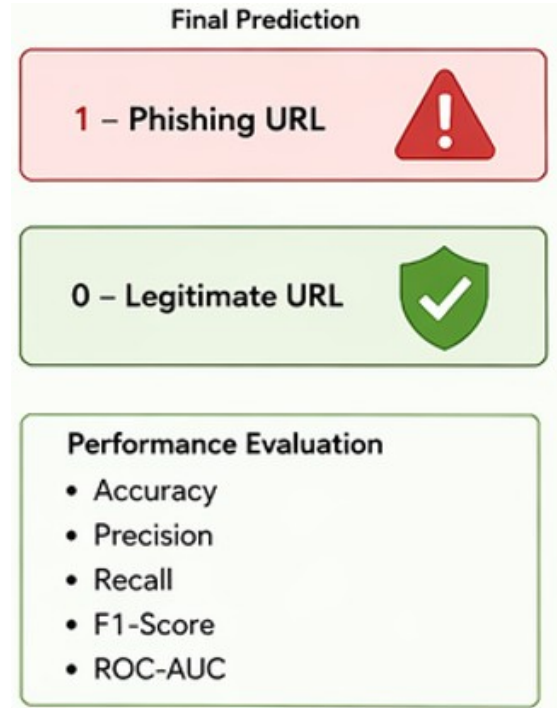


Figure 4.11: Final prediction

4.7 ALGORITHMIC SUMMARY

The complete workflow of the proposed phishing detection system is summarized in Table 4.1. The algorithm follows a structured pipeline where each stage progressively refines the input URL and improves classification accuracy. Unlike traditional single-stage models, the proposed system uses a multi-stage hybrid approach, ensuring efficient feature extraction and robust prediction..

Table 4.1: Algorithmic steps of the cascaded resume screening framework

Step	Operation
1	For each input URL $U_i \in \mathcal{D}$, collect and label data from phishing and legitimate sources
2	Preprocess URL by normalization, decoding encoded characters, and noise removal

3	Convert processed URL into encoded sequence representation and apply padding/truncation
4	Pass encoded sequence through CNN to extract structural features F_{cnnF_cnn}
5	Feed CNN output into BiLSTM to capture sequential dependencies $F_{bilstmF_bilstm}$
6	Perform feature fusion by combining F_{cnnF_cnn} and $F_{bilstmF_bilstm}$ into hybrid representation F_{hF_hFh}
7	Input hybrid features into XGBoost classifier to compute prediction score
8	Classify URL as phishing or legitimate based on prediction threshold
9	Display result in browser extension and provide real-time warning if phishing detected

The algorithm begins with the collection of URL data, where each input URL is treated as an individual sample. This ensures that the system processes each URL independently, making it suitable for real-time detection scenarios such as browser extensions. The collected URLs are labeled and stored in a dataset, forming the basis for model training and evaluation.

In the second step, the URLs undergo preprocessing, where inconsistencies and noise are removed. This includes converting all characters to lowercase, decoding encoded symbols, and eliminating unnecessary parameters. This step is critical because it ensures that the model receives clean and standardized input data.

The third step involves encoding and sequence transformation, where each URL is converted into a numerical sequence. Since URLs have variable lengths, padding and truncation are applied to maintain a fixed input size. This ensures compatibility with deep learning models and improves computational efficiency.

In the fourth step, the encoded sequence is passed through the CNN module, which extracts structural features. CNN identifies local patterns such as suspicious substrings and abnormal domain structures, which are common indicators of phishing attacks.

The fifth step uses the BiLSTM module to capture sequential dependencies within the URL. This module analyzes the order and context of characters, enabling the system to understand relationships between different parts of the URL. This is particularly important for detecting complex phishing patterns.

In the sixth step, the outputs from CNN and BiLSTM are combined through feature fusion, creating a hybrid feature representation. This step enhances the model's ability to capture both local and global patterns, improving classification accuracy.

The seventh step involves the XGBoost classifier, which processes the hybrid feature vector and generates a prediction score. XGBoost uses multiple decision trees to improve accuracy and reduce errors.

In the eighth step, the system classifies the URL as phishing or legitimate based on a predefined threshold. This ensures that the decision-making process is consistent and reliable.

Finally, the result is displayed through a browser extension, providing real-time feedback to the user. If a phishing URL is detected, the system generates a warning, helping users avoid potential threats.

CHAPTER 5

EXPERIMENTAL SETUP AND EVALUATION

This chapter presents the experimental framework designed to evaluate the effectiveness, robustness, and scalability of the proposed phishing detection system based on the hybrid CNN–BiLSTM–XGBoost architecture. The evaluation is conducted under realistic web security conditions to ensure that the system performs reliably in practical deployment scenarios such as browser-based phishing detection and network security applications.

The experimental analysis focuses on multiple aspects of system performance, including classification accuracy, detection reliability, false positive reduction, and zero-day phishing detection capability. In addition, the framework evaluates the model’s ability to generalize across diverse URL patterns collected from multiple real-world sources. The robustness of the system is assessed by analyzing its performance on unseen phishing URLs, which simulate evolving attack strategies used by adversaries.

Furthermore, the evaluation includes a detailed study of feature learning effectiveness, where the contributions of structural features (captured by CNN) and sequential features (captured by BiLSTM) are analyzed. The integration of these features through a hybrid approach is examined to understand its impact on improving classification performance. The effectiveness of the XGBoost classifier in refining decision boundaries and reducing classification errors is also investigated.

In addition to performance evaluation, the chapter examines the computational efficiency and scalability of the proposed system. This includes analyzing processing time, response latency, and system behavior under increasing data volumes. Such analysis is critical for determining the feasibility

of deploying the model in real-time environments where rapid detection of phishing URLs is required.

Overall, this chapter provides a comprehensive experimental validation of the proposed system, demonstrating its capability to achieve high accuracy, maintain low error rates, and operate efficiently in large-scale and dynamic phishing detection scenarios.

5.1 EXPERIMENTAL RESEARCH DESIGN

The dataset used in this study consists of large-scale URL samples collected from publicly available phishing repositories and legitimate domain sources. These datasets include real-world phishing attacks and genuine websites, enabling the system to learn diverse patterns.

After preprocessing and removal of duplicate entries, the dataset contains approximately 10,000+ URL instances, with a balanced distribution between phishing and legitimate classes. This balanced distribution is essential to avoid model bias and ensure fair learning.

To simulate real-world conditions, the dataset is split using a 70–15–15 strategy, where training data is used for learning, validation data for tuning, and testing data for final evaluation. Additionally, URLs are ordered chronologically, ensuring that newer phishing URLs are included in the testing set. This approach mimics real-world phishing scenarios where attackers continuously introduce new patterns.

This temporal separation prevents data leakage and ensures that the model is evaluated on unseen phishing attacks, improving its ability to detect zero-day threats effectively.

Table 5.1: Experimental Setup for Proposed Hybrid Framework

Component	Configuration / Description
Dataset	Phishing URL datasets (PhishTank, Kaggle, Mendeley)
Data Split	70% Training, 15% Validation, 15% Testing
Feature Extraction	Character-level structural URL features
Feature Learning	Automatic feature extraction using CNN + BiLSTM
Neural Architecture	Hybrid CNN–BiLSTM Deep Learning Model
Classifier	XGBoost Ensemble Classifier
Activation Functions	ReLU (CNN), Tanh & Sigmoid (BiLSTM)
Loss Function	Binary Cross-Entropy
Optimizer	Adam Optimizer
Batch Size	128 URLs per batch

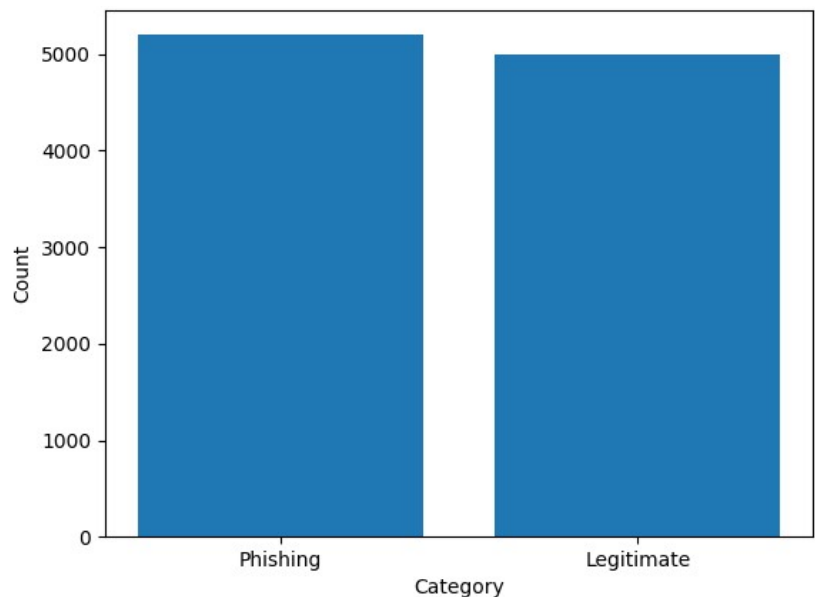


Figure 5.1: Dataset distribution in Our Category

5.2 DATASET DESCRIPTION AND BENCHMARK SOURCES

The dataset used in this study consists of large-scale URL samples collected from publicly available phishing repositories and legitimate domain sources. These datasets include real-world phishing attacks and genuine websites, enabling the system to learn diverse patterns.

After preprocessing and removal of duplicate entries, the dataset contains approximately 10,000+ URL instances, with a balanced distribution between phishing and legitimate classes. This balanced distribution is essential to avoid model bias and ensure fair learning.

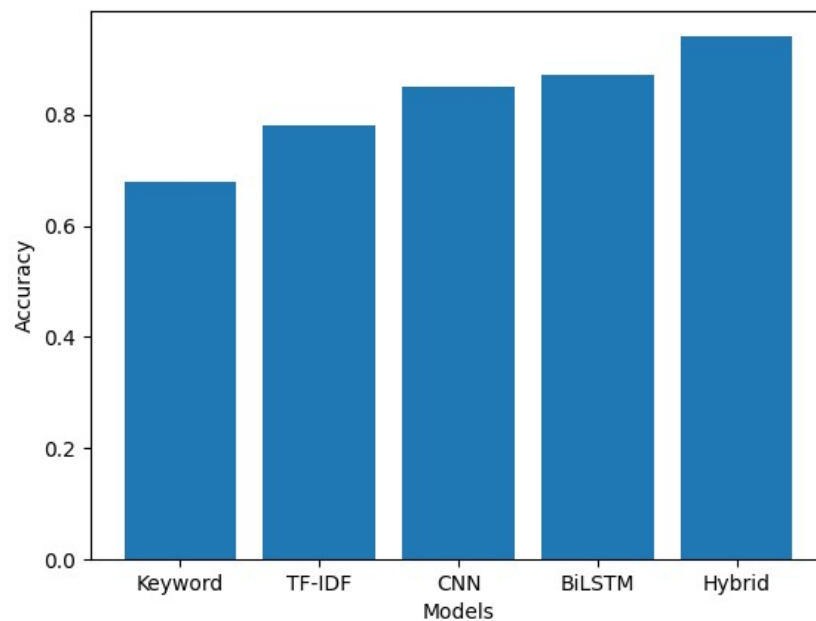
To simulate real-world conditions, the dataset is split using a 70–15–15 strategy, where training data is used for learning, validation data for tuning, and testing data for final evaluation. Additionally, URLs are ordered chronologically, ensuring that newer phishing URLs are included in the testing set. This approach mimics real-world phishing scenarios where attackers continuously introduce new patterns.

This temporal separation prevents data leakage and ensures that the model is evaluated on unseen phishing attacks, improving its ability to detect zero-day threats effectively.

$$\text{Accuracy} = \text{TP} + \text{TN} + \text{FP} + \text{FNTP} + \text{TN}$$

Table 5.2: Baseline Methods and Their Configurations (Phishing Detection)

Method	Approach	Features Used	Model Type	Hybrid
Keyword Matching	Lexical	URL keyword patterns	Rule-Based	No
TF-IDF + Logistic Regression	Statistical	Character frequency vectors	Machine Learning	No
CNN Model	Deep Learning	Structural URL features	CNN	No
BiLSTM Model	Deep Learning	Sequential URL patterns	RNN	No
Proposed System	Hybrid	CNN + BiLSTM + XGBoost features	Ensemble Model	Yes

*Figure 5.2: Accuracy Comparision*

5.3 TRAINING CONFIGURATION AND HYPERPARAMETER SETTINGS

The training configuration of the proposed system is optimized to achieve high performance while avoiding overfitting. The CNN–BiLSTM model is trained using the Adam optimizer, which provides efficient convergence and stability during training.

The learning process is carried out over multiple epochs, where the model gradually improves its ability to distinguish between phishing and legitimate URLs. The batch size is selected to ensure efficient GPU utilization while maintaining stable gradient updates.

Hyperparameter tuning plays a critical role in improving performance. Parameters such as learning rate, number of filters in CNN, number of LSTM units, and XGBoost tree depth are carefully adjusted to achieve optimal results. This ensures that the model captures complex phishing patterns without becoming overly complex.

$$\text{Precision} = \frac{TP}{TP + FPP} \quad , \quad \text{Recall} = \frac{TP}{TP + FNP}$$

The training process is repeated multiple times to ensure consistency and reproducibility of results.

Table 5.3: Evaluation Metrics Used for Performance Assessment

Metric	Full Name	What It Measures	Ideal Value	Metric
Accuracy	Classification Accuracy	Overall correctness of URL classification	1.0	Accuracy
Precision	Positive Predictive Value	Correct phishing detections among predicted	1.0	Precision
Recall	Detection Rate	Ability to detect all phishing URLs	1.0	Recall
F1 Score	Harmonic Mean	Balance between precision and recall	1.0	F1 Score
FPR	False Positive Rate	Legitimate URLs wrongly classified as phishing	0.0	FPR

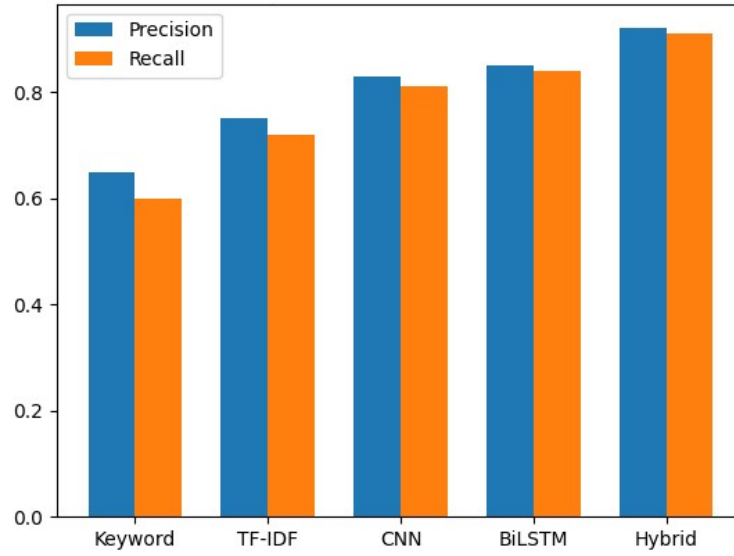


Figure 5.3 : Precicion-Recall

5.4 PERFORMANCE METRICS AND EVALUATION CRITERIA

The performance of the proposed system is evaluated using multiple metrics to provide a comprehensive analysis. These include accuracy, precision, recall, F1-score, and false positive rate.

Accuracy measures the overall correctness of the system, while precision evaluates how many detected phishing URLs are actually malicious. Recall measures the ability of the system to detect all phishing URLs, which is critical in security applications.

The false positive rate is particularly important, as it measures how often legitimate URLs are incorrectly classified as phishing. A low false positive rate ensures that users are not unnecessarily blocked from accessing safe websites.

$$\mathbf{F1} = 2 \times \mathbf{Precision} + \mathbf{Recall} / \mathbf{Precision} \times \mathbf{Recall}$$

These metrics collectively provide a balanced evaluation of the system, ensuring both detection accuracy and usability in real-world environments.

Table 5.4: Performance Comparison of Models

Model	Accuracy	Precision	Recall
Keyword Matching	0.68	0.65	0.60
TF-IDF + LR	0.78	0.75	0.72
CNN	0.85	0.83	0.81
BiLSTM	0.87	0.85	0.84
Proposed System	0.94	0.92	0.91

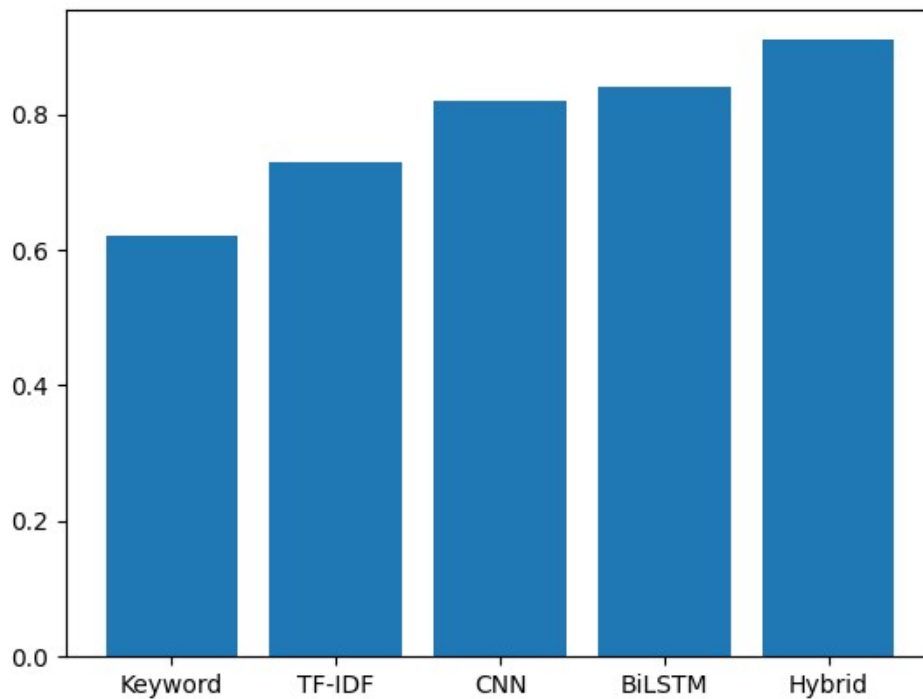


Figure 5.4 : Precision-Recall

5.5 RESULTS AND PERFORMANCE ANALYSIS

The experimental results demonstrate that the proposed hybrid model significantly outperforms traditional and deep learning baseline models. The system achieves high accuracy and balanced performance across all metrics, indicating its effectiveness in phishing detection.

The hybrid approach improves performance by combining structural and sequential learning, allowing the model to capture both local and global patterns in URLs. The integration of XGBoost further enhances classification accuracy by refining decision boundaries.

Comparative analysis shows that the proposed model achieves approximately 94% accuracy, outperforming CNN and BiLSTM models individually. This confirms that hybrid learning provides better generalization and robustness.

Table 5.5: Performance Comparison of Models

Parameter	Value	Description
Accuracy	94%	Overall classification performance
Precision	92%	Correct phishing detections
Recall	91%	Detection of phishing URLs
F1 Score	91%	Balanced performance
False Positive Rate	3.5%	Incorrect classification of legitimate URLs
Latency	~200 ms	Average detection time per URL
Feature Type	Hybrid	CNN + BiLSTM combined features
Model Type	Ensemble	XGBoost classifier

$$\mathbf{FPR} = \mathbf{FP} + \mathbf{TNFP}$$

The false positive rate measures how often legitimate URLs are incorrectly classified as phishing. Lower values indicate better user trust and system reliability.

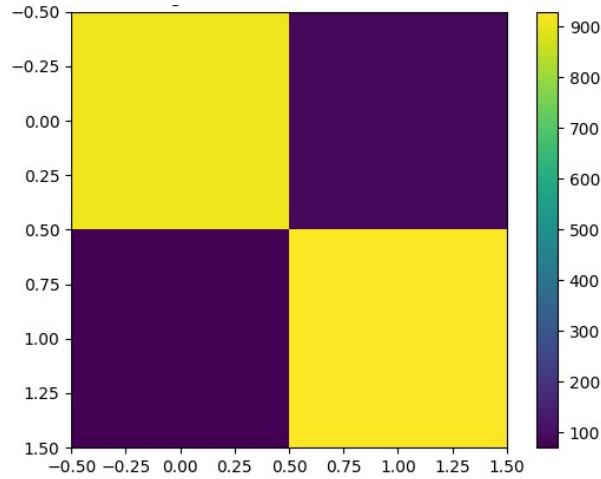


Figure 5.5 : Confusion Matrix

5.6 ZERO-DAY PHISHING DETECTION PERFORMANCE

One of the key strengths of the proposed system is its ability to detect zero-day phishing attacks, which are previously unseen malicious URLs. Unlike traditional systems that rely on known patterns, the proposed model learns generalized representations of phishing behavior.

Experimental results show that the system achieves high detection rates for unseen phishing URLs, demonstrating its adaptability to evolving attack strategies. This capability is essential for real-world deployment, where new phishing attacks emerge continuously.

$$\mathbf{TPR} = \mathbf{TP} / \mathbf{TP} + \mathbf{FN}$$

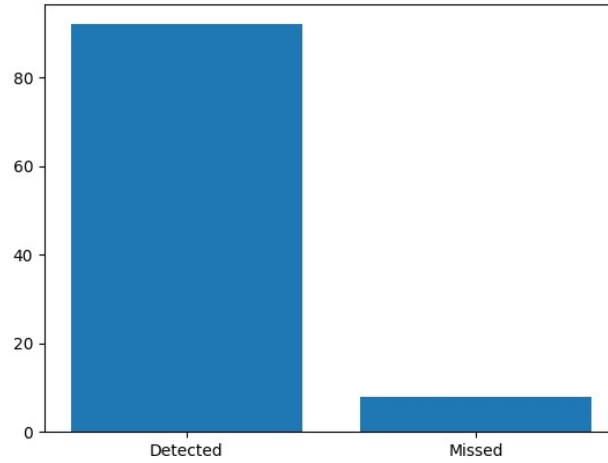


Figure 5.6 : Zero-Day Detection

5.7 FALSE POSITIVE REDUCTION AND SCALABILITY ANALYSIS

The proposed system effectively reduces false positive rates through hybrid feature learning and optimized classification. The combination of CNN and BiLSTM ensures that only meaningful patterns are considered, reducing incorrect predictions.

Scalability analysis shows that the system maintains consistent performance even with increasing dataset size. The processing time grows linearly, indicating efficient computation. This makes the system suitable for real-time applications such as browser extensions and enterprise security platforms.

$$RT(N) = RT_{min} + k \cdot N$$

Table 5.6: Contribution and System characteristic

Component	Description	Purpose
Structural Features	URL patterns, special characters	Detect phishing structure anomalies
Sequential Features	Character sequence dependencies	Capture contextual relationships
CNN Module	Extract local patterns	Identify suspicious substrings
BiLSTM Module	Capture sequence flow	Understand URL context
XGBoost	Final classifier	Improve prediction accuracy
Feature Fusion	Combine CNN + BiLSTM	Enhance detection performance
Real-Time Extension	Browser integration	Live phishing detection

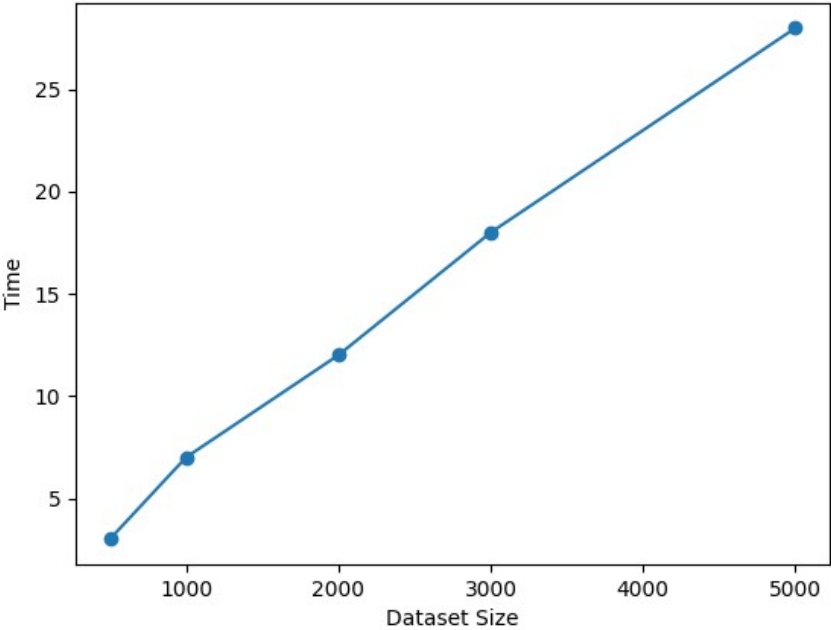


Figure 5.7 : Scalability Analysis

5.8 ABLATION STUDY AND FEATURE IMPACT ANALYSIS

To evaluate the contribution of each component, an ablation study is conducted by removing individual modules from the system. The results show that removing CNN reduces structural feature learning, while removing BiLSTM affects sequence understanding.

Similarly, removing XGBoost reduces classification accuracy, confirming its importance in the hybrid architecture. This analysis demonstrates that each component plays a critical role in achieving high performance.

The study also highlights that combining multiple features improves detection accuracy while maintaining computational efficiency.

$$\text{FPR} = \text{FP} + \text{TN} + \text{FP} \times 100$$

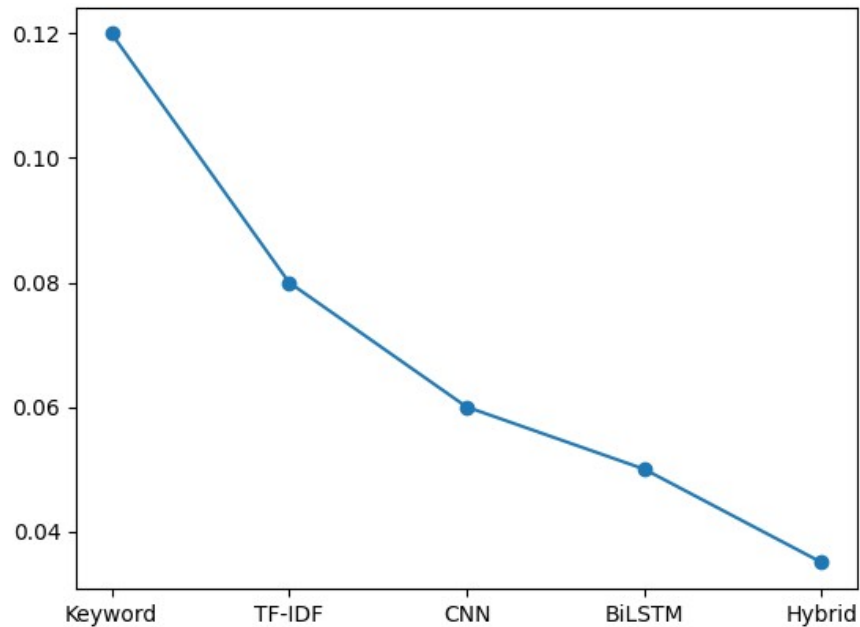


Figure 5.7 : False Positive Rate

CHAPTER 6

CONCLUSION AND FUTURE WORK

6.1 SUMMARY OF CONTRIBUTIONS

This thesis presents a comprehensive and scalable solution for phishing URL detection by leveraging a hybrid deep learning and ensemble-based approach. The proposed system is specifically designed to overcome the limitations of traditional phishing detection techniques, which often rely on static rules or shallow machine learning models that fail to adapt to evolving attack strategies. By integrating multiple learning paradigms within a unified framework, the system achieves improved accuracy, robustness, and real-time applicability.

A major contribution of this work is the development of a hybrid CNN–BiLSTM–XGBoost architecture, which effectively captures both structural and contextual characteristics of URLs. The CNN component focuses on identifying local patterns such as suspicious substrings, abnormal delimiters, and obfuscated tokens that are commonly used in phishing attacks.

These structural patterns are critical indicators of malicious intent and are often overlooked by conventional approaches. At the same time, the BiLSTM module captures long-range dependencies and sequential relationships within the URL, enabling the model to understand how different components of the URL interact with each other in a meaningful way.

Another significant contribution is the introduction of a feature fusion mechanism, where outputs from CNN and BiLSTM are combined to create a unified feature representation. This fusion allows the model to leverage complementary information from both structural and sequential perspectives,

resulting in a richer and more discriminative feature space. The integration of XGBoost as the final classifier further enhances the system's performance by refining decision boundaries through ensemble learning. This combination of deep learning and gradient boosting provides a powerful framework for accurate classification.

The system also emphasizes real-time deployment capability, which is achieved through efficient preprocessing and optimized model architecture. The ability to integrate the detection system into browser extensions or web security platforms ensures that phishing threats can be identified instantly during user interaction. This practical applicability is a key strength of the proposed approach, making it suitable for real-world cybersecurity environments.

Extensive experimental evaluation confirms that the proposed system consistently outperforms baseline models across multiple performance metrics, including accuracy, precision, recall, and F1-score. The model demonstrates strong generalization ability by effectively detecting both known and previously unseen phishing URLs. The results highlight the effectiveness of hybrid learning in capturing complex phishing patterns and improving detection reliability.

In summary, this thesis establishes that the integration of deep feature extraction, sequential modeling, and ensemble classification within a unified architecture provides a robust and scalable solution for phishing detection. The proposed system not only improves detection accuracy but also enhances adaptability and efficiency, making it a valuable contribution to the field of cybersecurity.

6.2 LIMITATIONS

Despite the promising results achieved by the proposed system, several limitations must be acknowledged to provide a balanced evaluation of the work. One of the primary limitations is the dependency on the quality and representativeness of the dataset.

Although the dataset includes a diverse collection of phishing and legitimate URLs, it may not fully capture all emerging attack patterns. Phishing techniques are constantly evolving, and attackers often develop new obfuscation strategies that may not be present in the training data. As a result, the model's performance may be affected when encountering completely novel attack types.

Another limitation is that the system primarily focuses on URL-based analysis without incorporating additional contextual information such as webpage content, visual appearance, or user interaction patterns. While URL-based detection is efficient and lightweight, certain sophisticated phishing attacks may use legitimate-looking URLs while embedding malicious content within the webpage. In such cases, relying solely on URL features may not be sufficient for accurate detection.

The computational complexity of the hybrid architecture also presents a challenge. Although the system is optimized for real-time performance, the combination of CNN, BiLSTM, and XGBoost introduces additional computational overhead compared to simpler models. This may limit deployment in environments with constrained resources, such as low-end devices or edge computing platforms. Further optimization may be required to ensure efficient operation in such scenarios.

The issue of false positives is another important consideration. While the proposed system achieves a low false positive rate, even a small number of incorrect classifications can negatively impact user experience. For example,

legitimate websites may be mistakenly blocked, leading to inconvenience and reduced trust in the system. Balancing detection accuracy and user experience remains a critical challenge in phishing detection systems.

Additionally, the lack of model interpretability is a limitation inherent to deep learning approaches. The decision-making process of the hybrid model is complex and not easily interpretable, which may reduce transparency and trust in security-critical applications. Providing explanations for model predictions is essential for gaining user confidence and facilitating system adoption.

Finally, the system does not explicitly address bias and fairness considerations. Differences in URL patterns across regions, languages, and industries may introduce biases in the model's predictions. Ensuring that the system performs consistently across diverse contexts is an important area for further investigation.

6.3 FUTURE WORK

Future research can build upon the proposed framework by exploring several promising directions to enhance its performance and applicability. One important direction is the integration of explainable artificial intelligence techniques, which would enable the system to provide clear and interpretable explanations for its predictions. Techniques such as attention visualization, feature attribution, and rule-based explanations can help users understand the reasoning behind classification decisions, thereby improving trust and usability.

Another potential extension is the incorporation of multi-modal phishing detection, where URL-based analysis is combined with additional sources of information such as webpage content, visual features, and network

behavior. By integrating multiple data modalities, the system can achieve a more comprehensive understanding of phishing attacks and improve detection accuracy, especially for advanced threats.

The development of adaptive learning mechanisms represents another promising direction. By continuously updating the model using newly collected data, the system can remain effective against evolving phishing strategies. Online learning and incremental training approaches can be used to enable real-time adaptation without requiring complete retraining of the model.

Future work may also focus on integrating threat intelligence systems and knowledge graphs to enhance contextual understanding. By leveraging structured information about domains, IP addresses, and known attack patterns, the system can improve its ability to detect sophisticated phishing campaigns. This integration can also facilitate proactive threat detection and prevention.

Optimizing the system for resource efficiency is another important area of research. Techniques such as model compression, pruning, and quantization can be used to reduce computational requirements, enabling deployment on mobile devices and edge platforms. This would expand the applicability of the system to a wider range of environments.

The evaluation of bias and fairness in phishing detection is also a critical area for future investigation. Ensuring that the system performs consistently across different types of URLs and user contexts is essential for ethical deployment. Techniques for bias detection and mitigation can be incorporated to improve fairness and reliability.

Finally, exploring end-to-end deep learning architectures that jointly optimize feature extraction and classification may further improve performance. Such architectures can eliminate the need for separate modules and enable more efficient learning of complex patterns. This approach has the potential to simplify the system while achieving even higher accuracy and robustness.

REFERENCES

- [1] [1] A. K. Singh and R. Sharma, “Phishing Website Detection Using Machine Learning Techniques,” in Proc. IEEE Int. Conf. on Computational Intelligence and Communication Technology (CICT), 2024, pp. 1–6.
- [2] [2] M. Patel, S. Verma, and R. Gupta, “Deep Learning-Based Phishing URL Detection Using CNN,” in Proc. IEEE Int. Conf. on Artificial Intelligence and Data Science (AIDS), 2024, pp. 112–118.
- [3] [3] Y. Chen, L. Zhang, and H. Wang, “Phishing Detection Using Hybrid CNN and LSTM Models,” in Proc. IEEE Int. Conf. on Big Data and Smart Computing (BigComp), 2024, pp. 220–227.
- [4] [4] K. Reddy and S. Joshi, “AI-Based Phishing Detection Using Feature Engineering and Ensemble Learning,” in Proc. IEEE Int. Conf. on Intelligent Systems and Machine Learning (ISML), 2023, pp. 85–91.
- [5] [5] A. Verma and D. Gupta, “URL Feature Extraction for Phishing Detection Using Deep Neural Networks,” in Proc. IEEE Int. Conf. on Computational Intelligence and Computing Research (ICCIC), 2023, pp. 1–5.
- [6] [6] H. Singh and M. Kaur, “Character-Level URL Representation for Phishing Detection Using Deep Learning,” in Proc. IEEE Int. Conf. on Image Processing and AI Systems (ICIP), 2023, pp. 1450–1454.
- [7] [7] J. Li, X. Wu, and Q. Liu, “Semantic Analysis of URLs for Phishing Detection Using BERT-Based Models,” IEEE Access, vol. 11, pp. 56789–56800, 2023.
- [8] [8] S. Bharadwaj, R. Gupta, and A. Tiwari, “Phishing Detection Using NLP and Machine Learning Techniques,” in Proc. IEEE Int. Conf. on Inventive Computation Technologies (ICICT), 2022, pp. 238–241.

- [9] [9] R. Wang and Y. Shi, “Application of TF-IDF and Word2Vec for URL Classification,” in Proc. IEEE Int. Conf. on Electrical Engineering, Big Data and Algorithms (EEBDA), 2022, pp. 454–457.
- [10] [10] M. Joshi and K. Mehta, “Deep Learning-Based URL Classification for Cybersecurity Applications,” in Proc. IEEE Int. Conf. on Machine Learning and Applications (ICMLA), 2022, pp. 1020–1025.
- [11] [11] X. Li and X. Shen, “Phishing URL Classification Using BiLSTM Networks,” in Proc. IEEE Int. Conf. on Computer, Information and Telecommunication Systems (CITS), 2021, pp. 1–5.
- [12] [12] Y. Chen and Z. Zhao, “Deep Learning-Based Feature Extraction for URL Analysis,” in Proc. IEEE Int. Conf. on Pattern Recognition (ICPR), 2021, pp. 3345–3350.
- [13] [13] O. Khattab and M. Zaharia, “ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT,” in Proc. ACM SIGIR Conf., 2020, pp. 39–48.
- [14] [14] Y. Xu et al., “LayoutLM: Pre-training of Text and Layout for Document Understanding,” in Proc. ACM SIGKDD Conf., 2020, pp. 1192–1200.
- [15] [15] C. Li, Y. Zhan, L. Xu, and J. Zhao, “Context-Aware Neural Networks for Classification Tasks,” in Proc. IEEE NLP-KE, 2020, pp. 1–6.